Object-Oriented and Classical Software Engineering

8th Edition, WCB/McGraw-Hill, 2010

Stephen R. Schach
srs@vuse.vanderbilt.edu

CHAPTER 12

OBJECT-ORIENTED ANALYSIS

Overview

- The analysis workflow
- Extracting the entity classes
- Object-oriented analysis: The elevator problem case study
- Functional modeling: The elevator problem case study
- Entity class modeling: The elevator problem case study
- Dynamic modeling: The elevator problem case study
- The test workflow: Object-oriented analysis

Overview (contd)

- Extracting the boundary and control classes
- The initial functional model: The MSG Foundation case study
- The initial class diagram: The MSG Foundation case study
- The initial dynamic model: The MSG Foundation case study
- Extracting the boundary classes: The MSG Foundation case study
- Extracting the boundary classes: The MSG Foundation case study

Overview (contd)

- Refining the use cases: The MSG Foundation case study
- Use-case realization: The MSG Foundation case study
- Incrementing the class diagram: The MSG Foundation case study
- The specification document in the Unified Process
- More on actors and use cases
- CASE tools for the object-oriented analysis workflow
- Challenges of the object-oriented analysis workflow

Object-Oriented Analysis

- OOA is a semiformal analysis technique for the object-oriented paradigm
  - There are over 60 equivalent techniques
  - Today, the Unified Process is the only viable alternative
- During this workflow
  - The classes are extracted
- Remark
  - The Unified Process assumes knowledge of class extraction

12.1 The Analysis Workflow

- The analysis workflow has two aims
  - Obtain a deeper understanding of the requirements
  - Describe them in a way that will result in a maintainable design and implementation

The Analysis Workflow (contd)

- There are three types of classes:
- Entity classes
- Boundary classes
- Control classes

The Analysis Workflow (contd)

- Entity class
  - Models long-lived information
- Examples:
  - **Account Class**
  - **Investment Class**

The Analysis Workflow (contd)

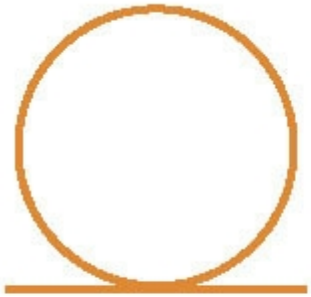
- Boundary class
  - Models the interaction between the product and the environment
  - A boundary class is generally associated with input or output

- Examples:
  - **Investments Report Class**
  - **Mortgages Report Class**

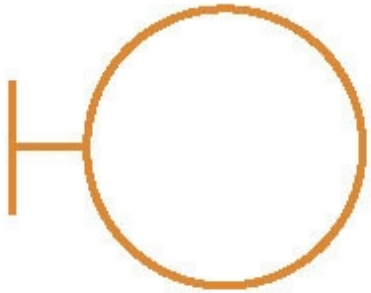
The Analysis Workflow (contd)

- Control class
  - Models complex computations and algorithms
- Example:
  - **Estimate Funds for Week Class**

UML Notation for These Three Class Types



Entity Class



Boundary Class



Control Class

12.2 Extracting the Entity Classes

- Perform the following three steps incrementally and iteratively
  - Functional modeling
    - Present scenarios of all the use cases (a *scenario* is an instance of a use case)
  - Class modeling
    - Determine the entity classes and their attributes
    - Determine the interrelationships and interactions between the entity classes
    - Present this information in the form of a *class diagram*
  - Dynamic modeling
    - Determine the operations performed by or to each entity class
    - Present this information in the form of a *statechart*

A product is to be installed to control n elevators in a building with m floors. The problem concerns the logic required to move elevators between floors according to the following constraints:

1. Each elevator has a set of m buttons, one for each floor. These illuminate when pressed and cause the elevator to visit the corresponding floor. The illumination is canceled when the corresponding floor is visited by the elevator

2. Each floor, except the first and the top floor, has two buttons, one to request an up-elevator, one to request a down-elevator. These buttons illuminate when pressed. The illumination is canceled when an elevator visits the floor, then moves in the desired direction

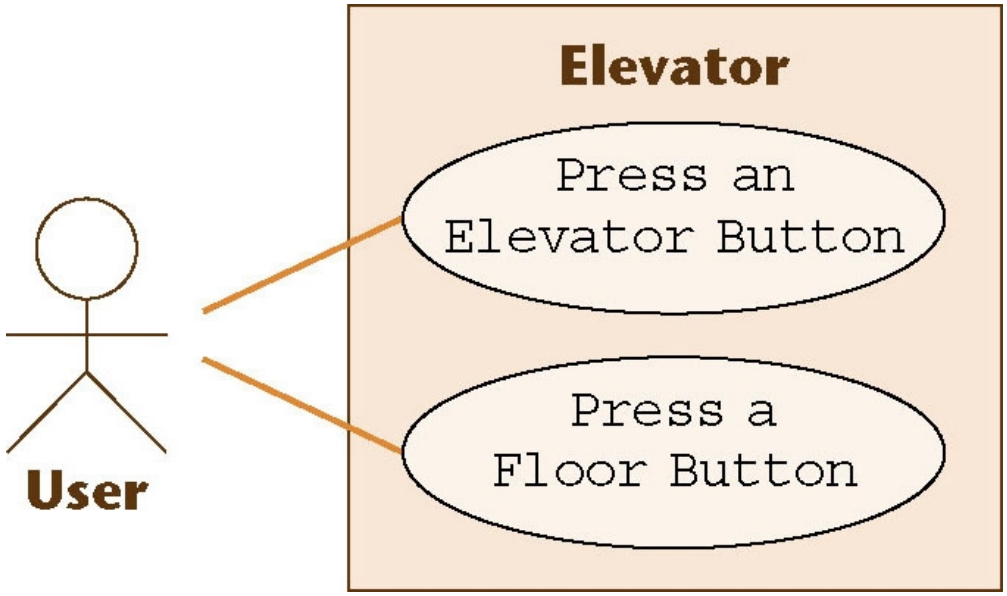
3. If an elevator has no requests, it remains at its current floor with its doors closed

12.4 Functional Modeling: The Elevator Problem Case Study

- A use case describes the interaction between
  - The product, and
  - The actors (external users)

Use Cases

- For the elevator problem, there are only two possible use cases
  - `Press an Elevator Button`, and
  - `Press a Floor Button`



Scenarios

- A use case provides a generic description of the overall functionality
- A scenario is an instance of a use case
- Sufficient scenarios need to be studied to get a comprehensive insight into the target product being modeled

Normal Scenario: Elevator Problem

1. User A presses the Up floor button at floor 3 to request an elevator. User A wishes to go to floor 7.
2. The Up floor button is turned on.
3. An elevator arrives at floor 3. It contains User B, who has entered the elevator at floor 1 and pressed the elevator button for floor 9.
4. The elevator doors open.
5. The timer starts.
   User A enters the elevator.
6. User A presses the elevator button for floor 7.
7. The elevator button for floor 7 is turned on.
8. The elevator doors close after a timeout.
9. The Up floor button is turned off.
10. The elevator travels to floor 7.
11. The elevator button for floor 7 is turned off.
12. The elevator doors open to allow User A to exit from the elevator.
13. The timer starts.
    User A exits from the elevator.
14. The elevator doors close after a timeout.
15. The elevator proceeds to floor 9 with User B.

Exception Scenario: Elevator Problem

1. User A presses the Up floor button at floor 3 to request an elevator. User A wishes to go to floor 1.
2. The Up floor button is turned on.
3. An elevator arrives at floor 3. It contains User B, who has entered the elevator at floor 1 and pressed the elevator button for floor 9.
4. The elevator doors open.
5. The timer starts.
   User A enters the elevator.
6. User A presses the elevator button for floor 1.
7. The elevator button for floor 1 is turned on.
8. The elevator doors close after a timeout.
9. The Up floor button is turned off.
10. The elevator travels to floor 9.
11. The elevator button for floor 9 is turned off.
12. The elevator doors open to allow User B to exit from the elevator.
13. The timer starts.
    User B exits from the elevator.
14. The elevator doors close after a timeout.
15. The elevator proceeds to floor 1 with User A.

12.5 Entity Class Modeling : The Elevator Problem Case Study

- Extract classes and their attributes
  - Represent them using a UML diagram
- One alternative: Deduce the classes from use cases and their scenarios
  - Possible danger: Often there are many scenarios, and hence
  - Too many candidate classes
- Other alternatives:
  - CRC cards (if you have domain knowledge)
  - Noun extraction

12.5.1 Noun Extraction

- A two-stage process

- Stage 1. Concise problem definition
  - Describe the software product in single paragraph
  - Buttons in elevators and on the floors control the movement of n elevators in a building with m floors. Buttons illuminate when pressed to request the elevator to stop at a specific floor; the illumination is canceled when the request has been satisfied. When an elevator has no requests, it remains at its current floor with its doors closed

Noun Extraction (contd)

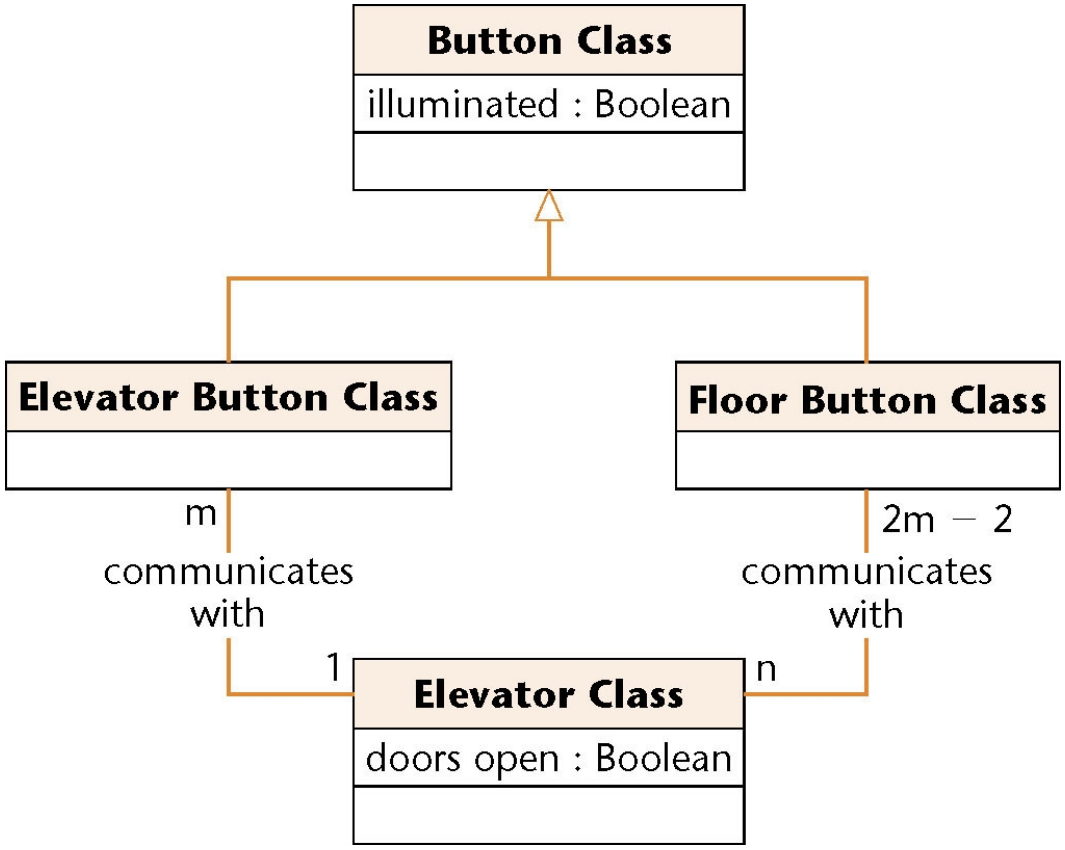
- Stage 2. Identify the nouns
  - Identify the nouns in the informal strategy
  - Buttons in elevators and on the floors control the movement of n elevators in a building with m floors. Buttons illuminate when pressed to request the elevator to stop at a specific floor; the illumination is canceled when the request has been satisfied. When an elevator has no requests, it remains at its current floor with its doors closed

- Use the nouns as candidate classes

Noun Extraction (contd)

- Nouns
  - button, elevator, floor, movement, building, illumination, request, door
  - floor, building, door are outside the problem boundary — exclude
  - movement, illumination, request are abstract nouns — exclude (they may become attributes)
- Candidate classes:
  - **Elevator Class** and **Button Class**
- Subclasses:
  - **Elevator Button Class** and **Floor Button Class**

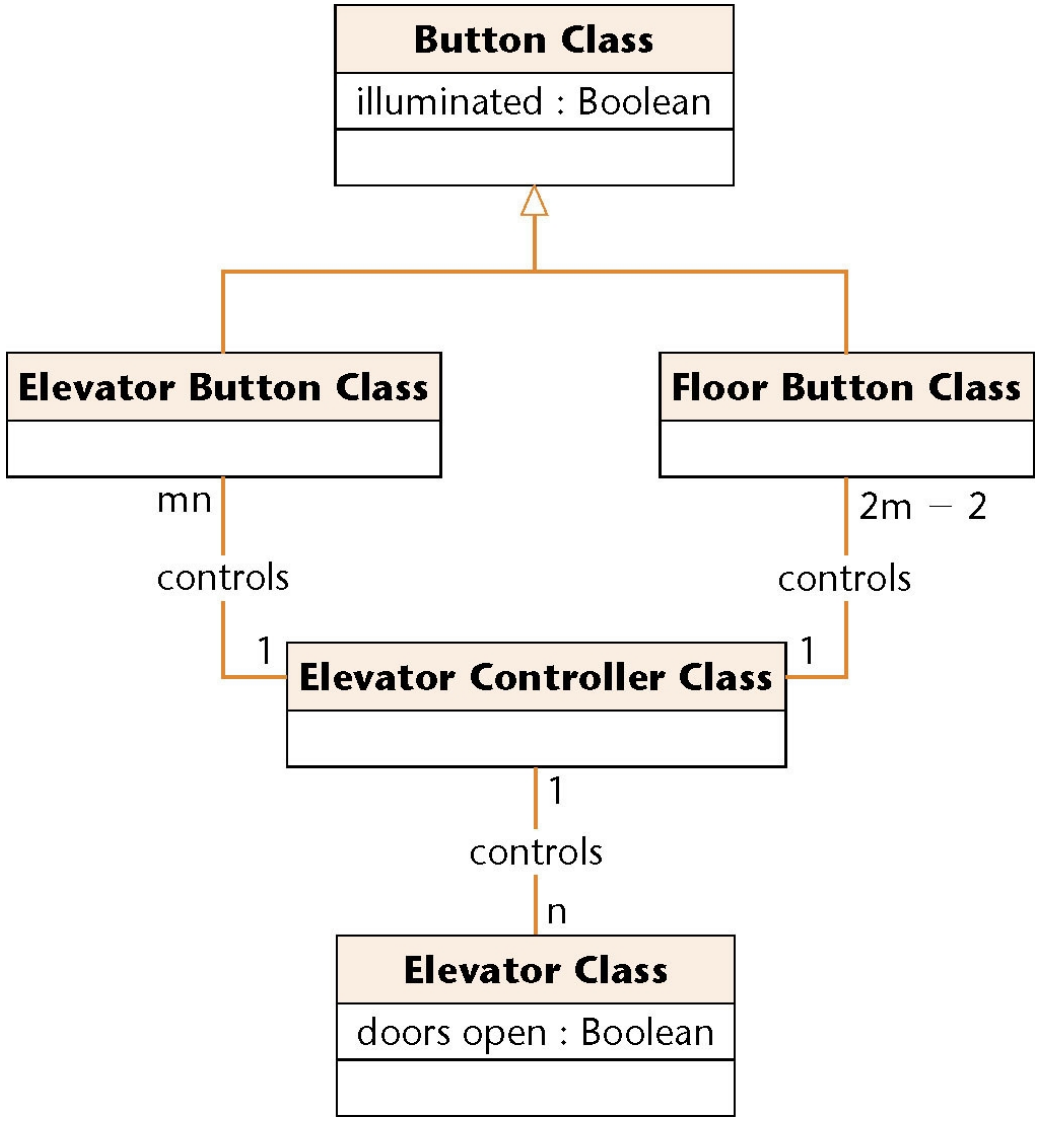
First Iteration of Class Diagram



- Problem
  - Buttons do not communicate directly with elevators
  - We need an additional class: **Elevator Controller Class**

Second Iteration of Class Diagram

- All relationships are now 1-to-n
  - This makes design and implementation easier



12.5.2 CRC Cards

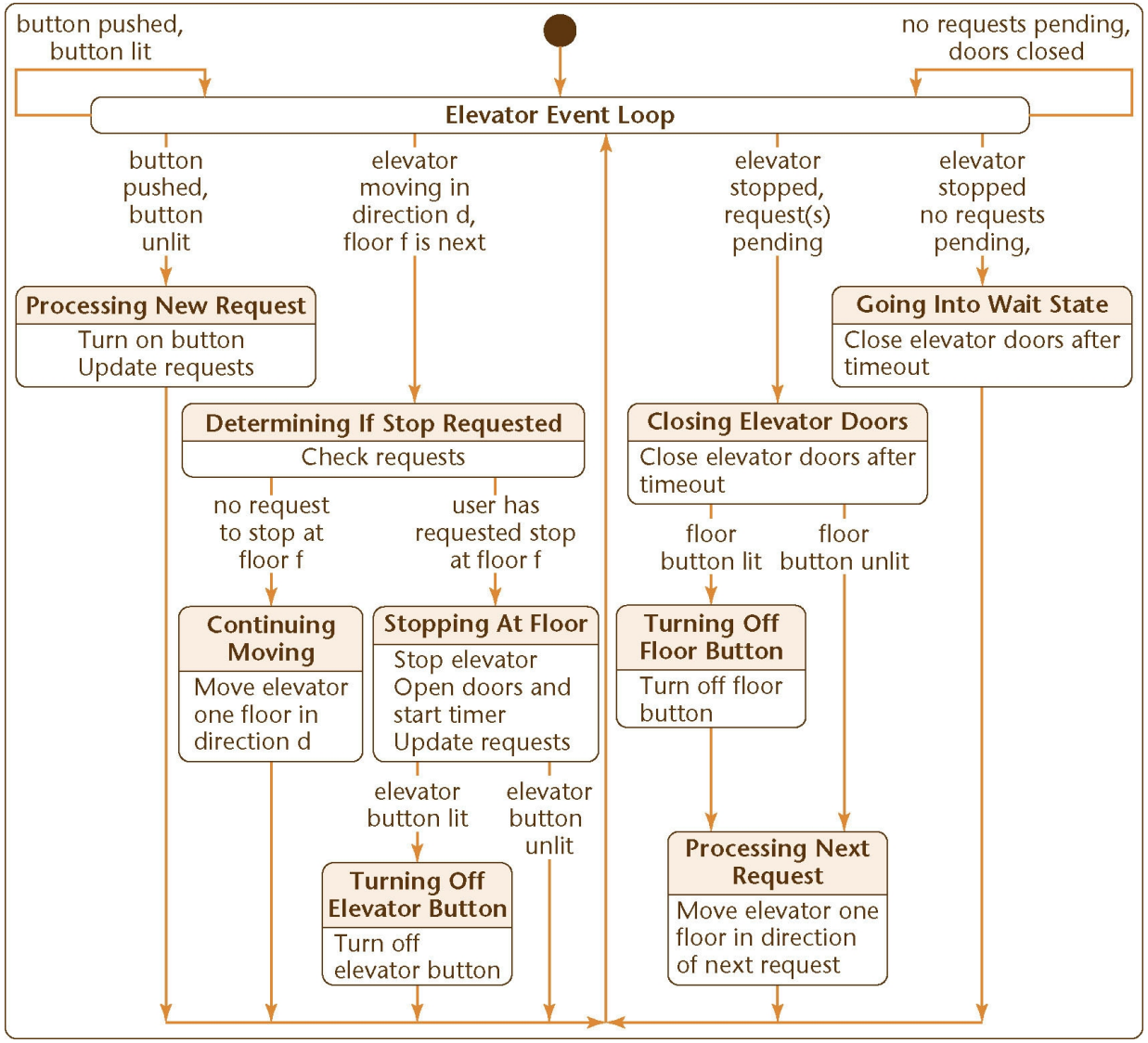
- Used since 1989 for OOA
- For each class, fill in a card showing
  - Name of Class
  - Functionality (Responsibility)
  - List of classes it invokes (Collaboration)
- Now CRC cards are automated (CASE tool component)

CRC Cards (contd)

- Strength
  - When acted out by team members, CRC cards are a powerful tool for highlighting missing or incorrect items
- Weakness
  - If CRC cards are used to identify entity classes, domain expertise is needed

12.6 Dynamic Modeling: The Elevator Problem Case Study

- Produce a UML statechart
- State, event, and predicate are distributed over the statechart



Dynamic Modeling: Elevator Problem (contd)

- This UML statechart is equivalent to the state transition diagram of Figures 11.15 through 11.17
- This is shown by considering specific scenarios
- In fact, a statechart is constructed by modeling the events of the scenarios

12.7 The Test Workflow: Object-Oriented Analysis

- CRC cards are an excellent testing technique

CLASS **Elevator Controller Class**
RESPONSIBILITY 1. Turn on elevator button 2. Turn off elevator button 3. Turn on floor button 4. Turn off floor button 5. Move elevator up one floor 6. Move elevator down one floor 7. Open elevator doors and start timer 8. Close elevator doors after timeout 9. Check requests 10. Update requests
COLLABORATION 1. **Elevator Button Class** 2. **Floor Button Class** 3. **Elevator Class**

CRC Cards

- Consider responsibility
  - 1. Turn on elevator button

- This is totally inappropriate for the object-oriented paradigm
  - Responsibility-driven design has been ignored
  - Information hiding has been ignored

- Responsibility
  1. Turn on elevator button

  should be

  1. Send message to **Elevator Button Class** to turn itself on

CRC Cards (contd)

- Also, a class has been overlooked
- The elevator doors have a *state* that changes during execution (class characteristic)
  - Add class **Elevator Doors Class**
  - Safety considerations
- Modify the CRC card

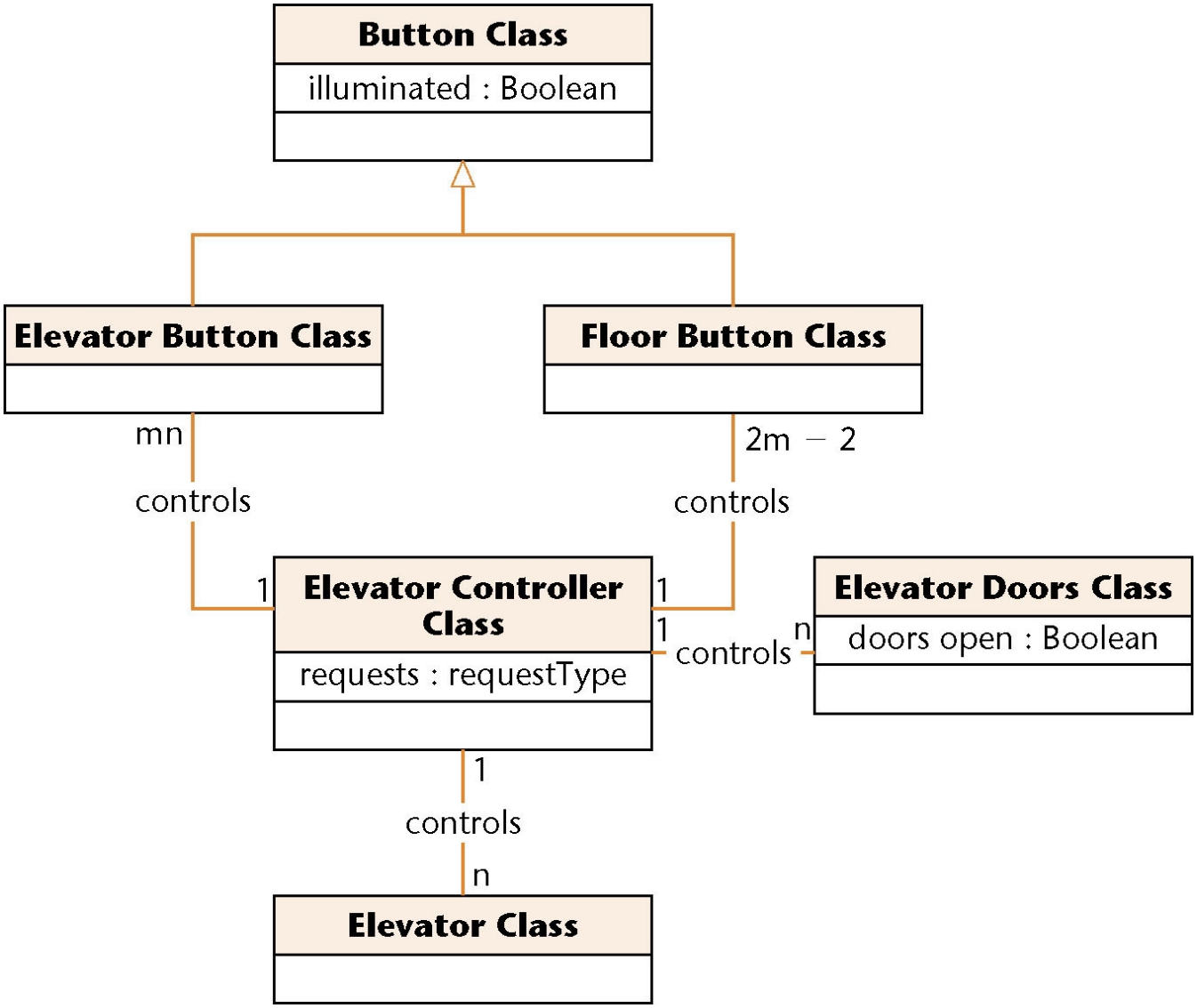
Second Iteration of the CRC Card

CLASS **Elevator Controller Class**
RESPONSIBILITY 1. Send message to **Elevator Button Class** to turn on button 2. Send message to **Elevator Button Class** to turn off button 3. Send message to **Floor Button Class** to turn on button 4. Send message to **Floor Button Class** to turn off button 5. Send message to **Elevator Class** to move up one floor 6. Send message to **Elevator Class** to move down one floor 7. Send message to **Elevator Doors Class** to open 8. Start timer 9. Send message to **Elevator Doors Class** to close after timeout 10. Check requests 11. Update requests
COLLABORATION 1. **Elevator Button Class** (subclass) 2. **Floor Button Class** (subclass) 3. **Elevator Doors Class** 4. **Elevator Class**

CRC Cards (contd)

- Having modified the class diagram, reconsider the
  - Use-case diagram (no change)
  - Class diagram (see the next slide)
  - Statecharts
  - Scenarios (see the slide after the next slide)

Third Iteration of Class Diagram



Second Iteration of the Normal Scenario:

1. User A presses the Up floor button at floor 3 to request an elevator. User A wishes to go to floor 7.
2. The floor button informs the elevator controller that the floor button has been pushed.
3. The elevator controller sends a message to the Up floor button to turn itself on.
4. The elevator controller sends a series of messages to the elevator to move itself up to floor 3. The elevator contains User B, who has entered the elevator at floor 1 and pressed the elevator button for floor 9.
5. The elevator controller sends a message to the elevator doors to open themselves.
6. The elevator control starts the timer.
   User A enters the elevator.
7. User A presses elevator button for floor 7.
8. The elevator button informs the elevator controller that the elevator button has been pushed.
9. The elevator controller sends a message to the elevator button for floor 7 to turn itself on.
10. The elevator controller sends a message to the elevator doors to close themselves after a timeout.
11. The elevator controller sends a message to the Up floor button to turn itself off.
12. The elevator controller sends a series of messages to the elevator to move itself up to floor 7.
13. The elevator controller sends a message to the elevator button for floor 7 to turn itself off.
14. The elevator controller sends a message to the elevator doors to open themselves to allow User A to exit from the elevator.
15. The elevator controller starts the timer.
    User A exits from the elevator.
16. The elevator controller sends a message to the elevator doors to close themselves after a timeout.
17. The elevator controller sends a series of messages to the elevator to move itself up to floor 9 with User B.

OOA: Elevator Problem (contd)

- The object-oriented analysis is now fine
- We should rather say:
  - The object-oriented analysis is fine *for now*
- We may need to return to the object-oriented analysis workflow during the object-oriented design workflow

12.8 Extracting the Boundary and Control Classes

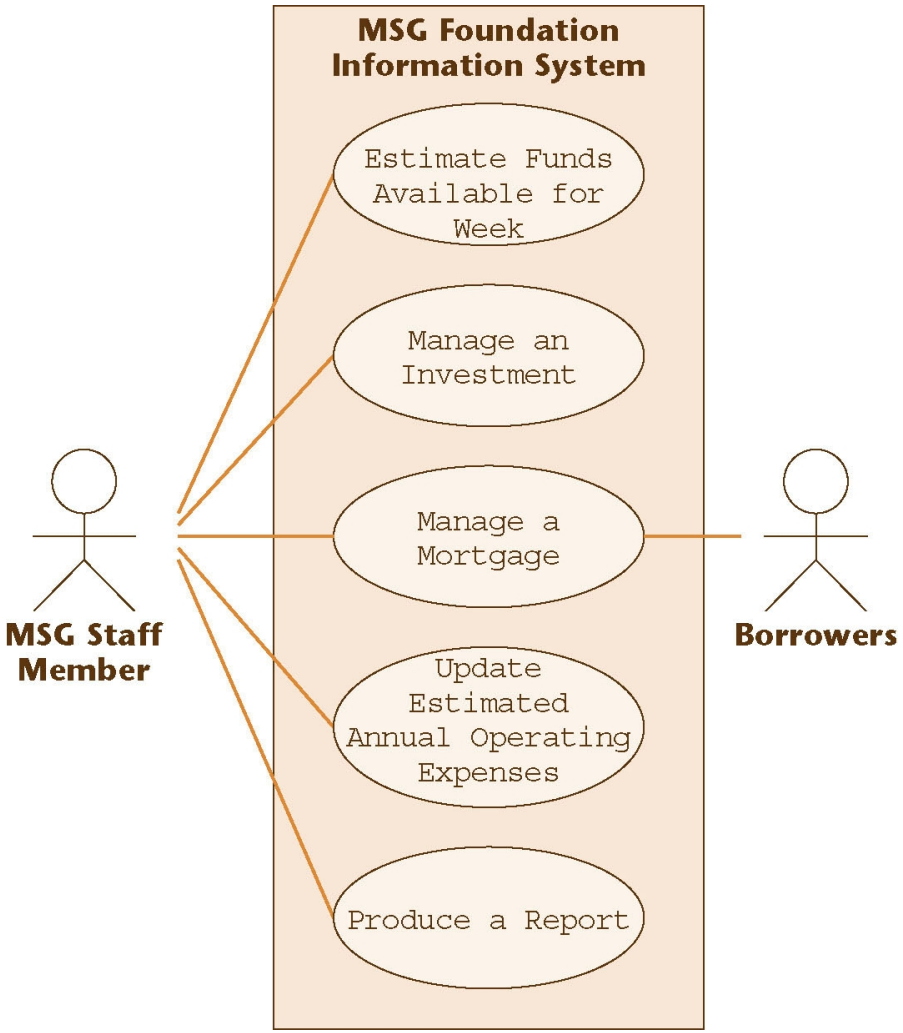
- Each
  - Input screen,
  - Output screen, and
  - Report

  is modeled by its own boundary class

- Each nontrivial computation is modeled by a control class

12.9 The Initial Functional Model: MSG Foundation

- Recall the seventh iteration of the use-case diagram



Use Case Manage a Mortgage

- One possible extended scenario

An MSG Foundation staff member wants to update the annual real-estate tax on a home for which the Foundation has provided a mortgage.

1. The staff member enters the new value of the annual real-estate tax.
2. The information system updates the date on which the annual real-estate tax was last changed.

Possible Alternative

A. The staff member enters the mortgage number incorrectly.

Use Case `Manage a Mortgage` (contd)

- A second extended scenario

There is a change in the weekly income of a couple who have borrowed money from the MSG Foundation. They wish to have their weekly income updated in the Foundation records by an MSG staff member so that their mortgage payments will be correctly computed.

1. The staff member enters the new value of the weekly income.
2. The information system updates the date on which the weekly income was last changed.

Possible Alternatives

A. The staff member enters the mortgage number incorrectly.
B. The borrowers do not bring documentation regarding their new income.

Use Case Estimate Funds Available for Week

- One possible scenario

An MSG Foundation staff member wishes to determine the funds available for mortgages this week.

1. For each investment, the information system extracts the estimated annual return on that investment. It sums the separate returns and divides the result by 52 to yield the estimated investment income for the week.
2. The information system then extracts the estimated annual MSG Foundation operating expenses and divides the result by 52.
3. For each mortgage:
   - 3.1 The information system computes the amount to be paid this week by adding the principal and interest payment to $\frac{1}{52}$nd of the sum of the annual real-estate tax and the annual homeowner's insurance premium.
   - 3.2 It then computes 28 percent of the couple's current gross weekly income.
   - 3.3 If the result of Step 3.1 is greater than the result of Step 3.2, then it determines the mortgage payment for the week as the result of Step 3.2, and the amount of the grant for this week as the difference between the result of Step 3.1 and the result of Step 3.2.
   - 3.4 Otherwise, it takes the mortgage payment for this week as the result of Step 3.1, and there is no grant for the week.
4. The information system sums the mortgage payments of Steps 3.3 and 3.4 to yield the estimated total mortgage payments for the week.
5. It sums the grant payments of Step 3.3 to yield the estimated total grant payments for the week.
6. The information system adds the results of Steps 1 and 4 and subtracts the results of Steps 2 and 5. This is the total amount available for mortgages for the current week.
7. Finally, the software product prints the total amount available for new mortgages during the current week.

Use Case `Produce a Report`

- One possible scenario

An MSG staff member wishes to print a list of all mortgages.

1. The staff member requests a report listing all mortgages.

Use Case `Produce a Report` (contd)

- Another possible scenario

An MSG staff member wishes to print a list of all investments.

1. The staff member requests a report listing all investments.

12.10 The Initial Class Diagram: MSG Foundation

- The aim of entity modeling step is to extract the entity classes, determine their interrelationships, and find their attributes

- Usually, the best way to begin this step is to use the two-stage noun extraction method

Noun Extraction: MSG Foundation

- Stage 1: Describe the information system in a single paragraph
  - Weekly reports are to be printed showing how much money is available for mortgages. In addition, lists of investments and mortgages must be printed on demand.

Noun Extraction: MSG Foundation (contd)

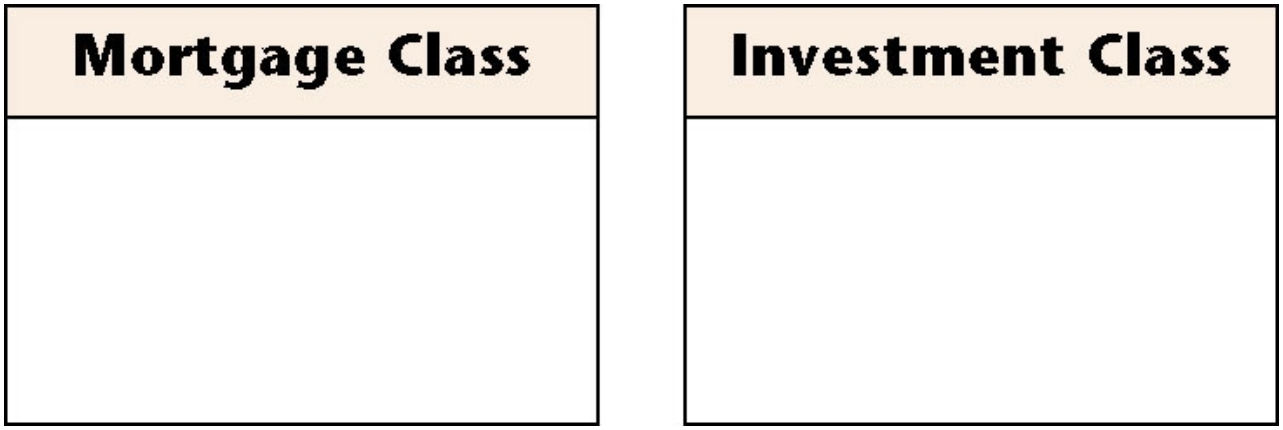
- Stage 2: Identify the nouns in this paragraph
  - Weekly reports are to be printed showing how much money is available for mortgages. In addition, lists of investments and mortgages must be printed on demand.

- The nouns are report, money, mortgage, list, and investment

Noun Extraction: MSG Foundation (contd)

- Nouns report and list are not long lived, so they are unlikely to be entity classes (report will surely turn out to be a boundary class)
- money is an abstract noun
- This leaves two candidate entity classes
  - **Mortgage Class** and **Investment Class**

First Iteration of the Initial Class Diagram

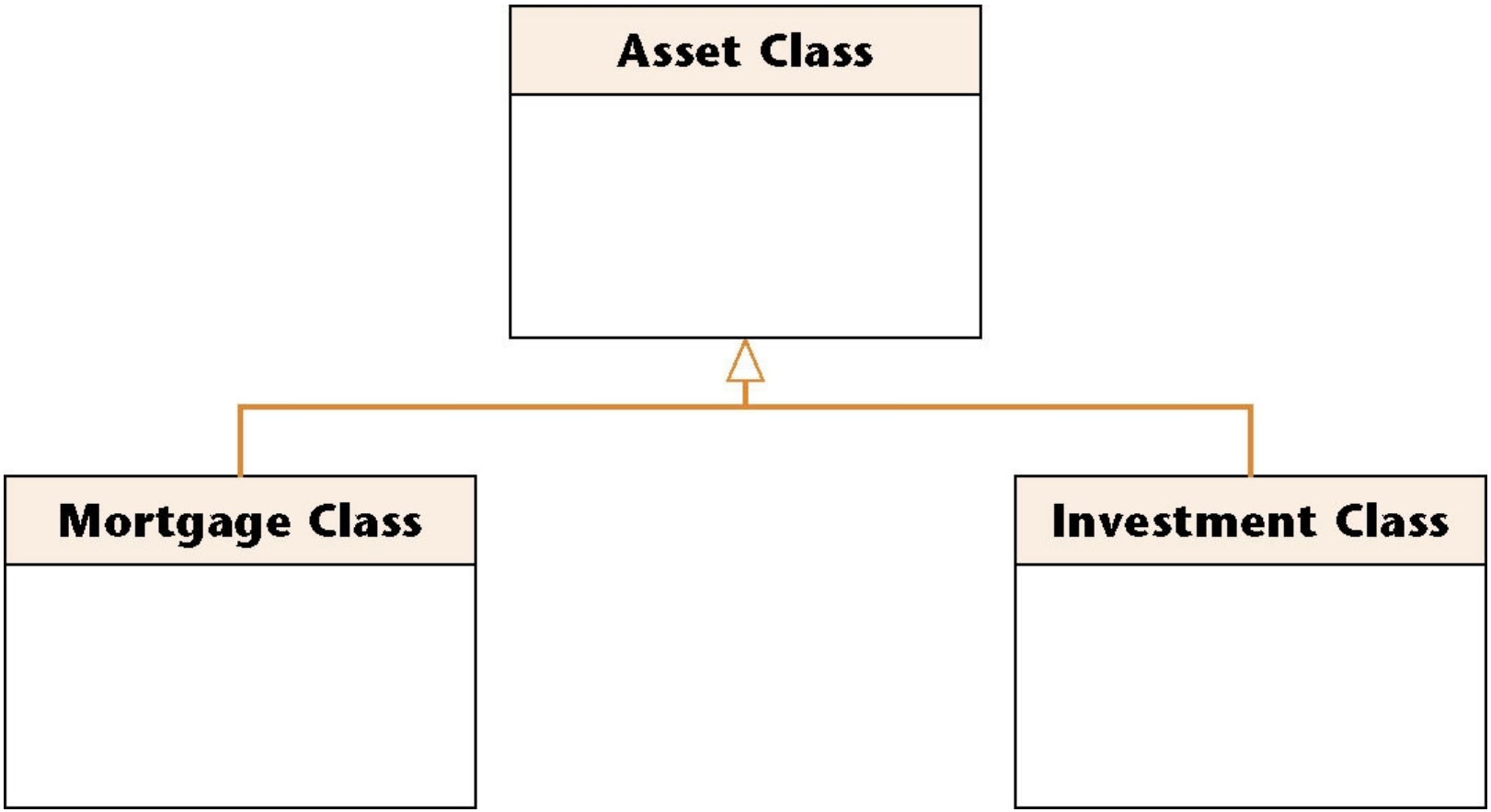


Second Iteration of the Initial Class Diagram

- Operations performed on the two entity classes are likely to be very similar
  - Insertions, deletions, and modifications
  - All members of both entity classes have to be printed on demand

- **Mortgage Class** and **Investment Class** should be subclasses of a superclass called **Asset Class**

Second Iteration of Initial Class Diagram (contd)

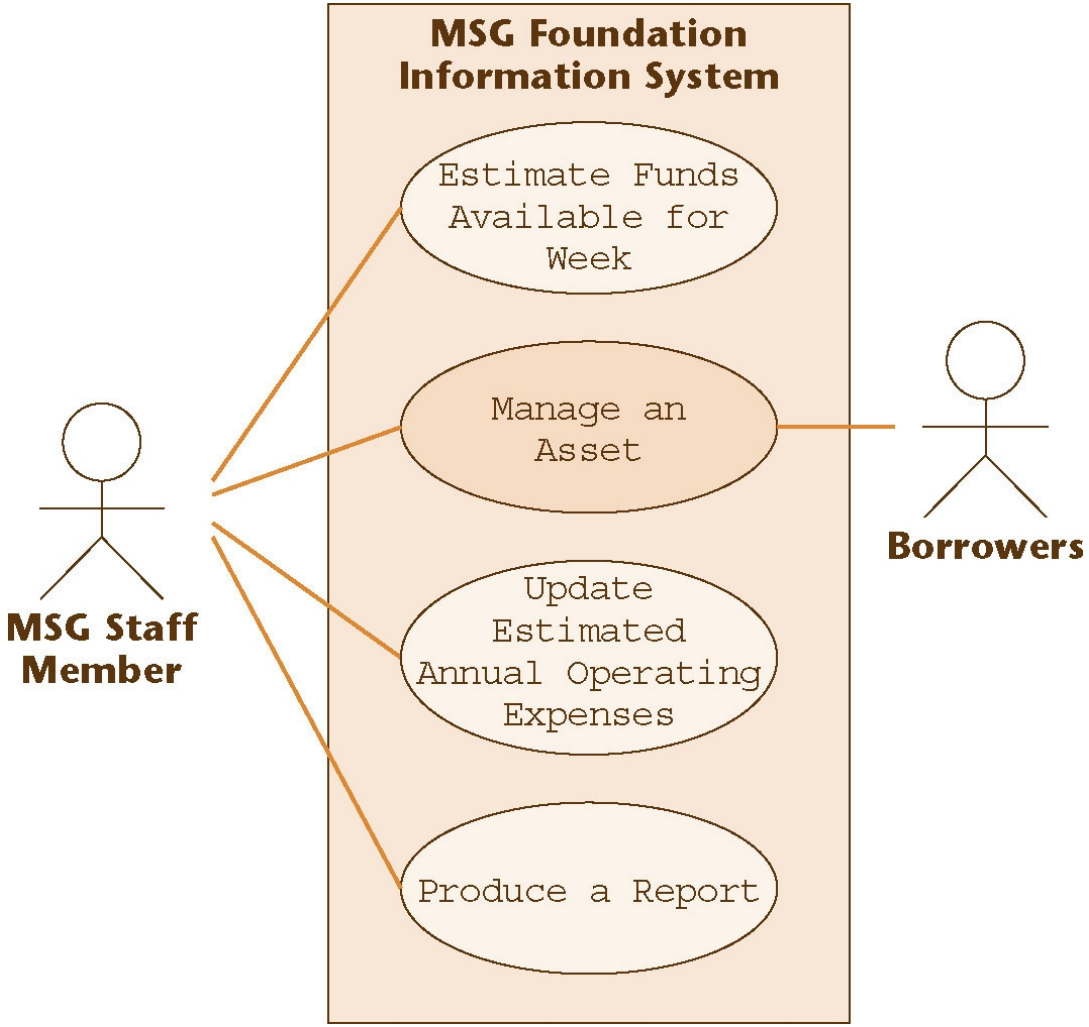


Back to the Requirements Workflow

- The current five use cases include `Manage a Mortgage` and `Manage an Investment`
- These two can now be combined into a single use case, `Manage an Asset`

Eighth Iteration of the Use-Case Diagram

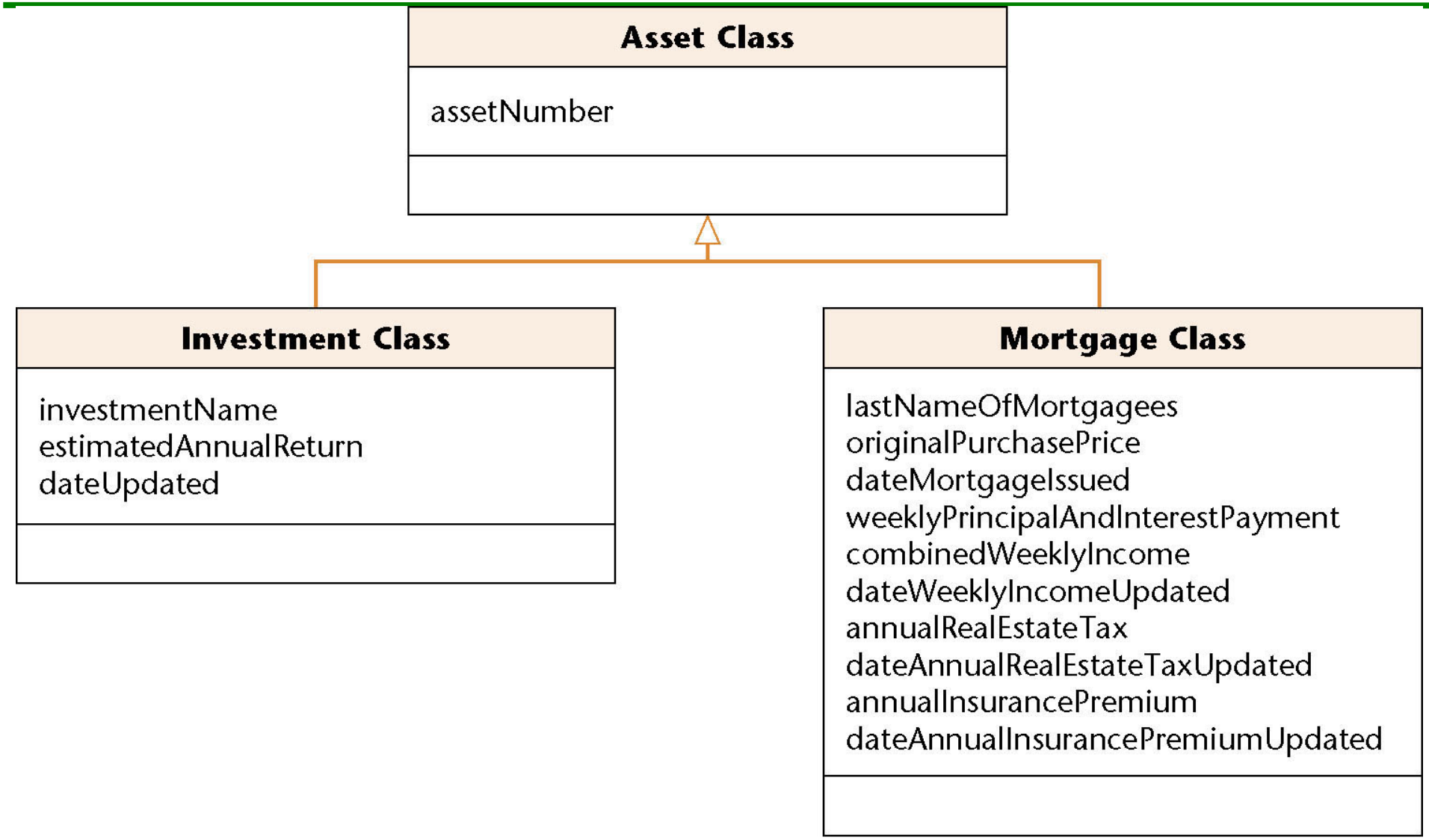
- The new use case is shaded



Initial Class Diagram: MSG Foundation (contd)

- Finally, we add the attributes of each class to the class diagram
  - For the MSG Foundation case study, the result is shown on the next slide
- The empty rectangle at the bottom of each box will later be filled with the operations of that class

Second Iteration of Initial Class Diagram (contd)



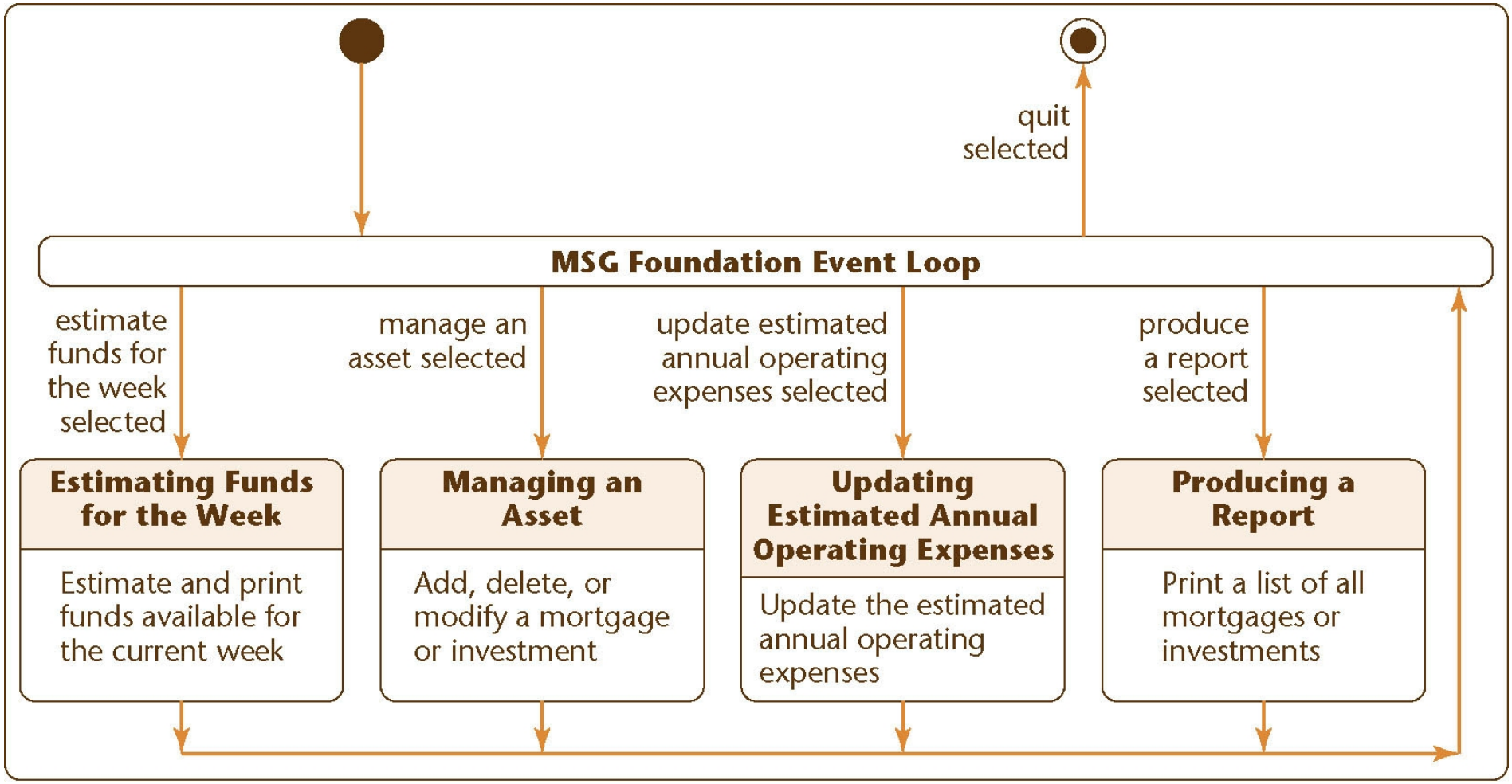
Iteration and Incrementation

- The phrase “iterate and increment” also includes the possibility of having to decrement what has been developed to date
  - A mistake may have been made, and backtracking is needed
  - As a consequence of reorganizing the UML models, one or more artifacts may have become superfluous

12.11 The Initial Dynamic Model: MSG Foundation

- Dynamic modeling is the third step in extracting the entity classes
- A statechart is constructed that reflects all the operations performed by or to the software product
- The operations are determined from the scenarios

Initial Dynamic Model: MSG Foundation (contd)



Initial Dynamic Model: MSG Foundation (contd)

- The statechart reflects the operations of the complete MSG Foundation information system
  - The solid circle on the top left represents the initial state, the starting point of the statechart
  - The white circle containing the small black circle on the top right represents the final state
  - States other than the initial and final states are represented by rectangles with rounded corners
  - The arrows represent possible transitions from state to state

Initial Dynamic Model: MSG Foundation (contd)

- In state **MSG Foundation Information System Loop**, one of five events can occur

- An MSG staff member can issue one of five commands:
  - estimate funds for the week
  - manage an asset
  - update estimated annual operating expenses
  - produce a report, or
  - quit

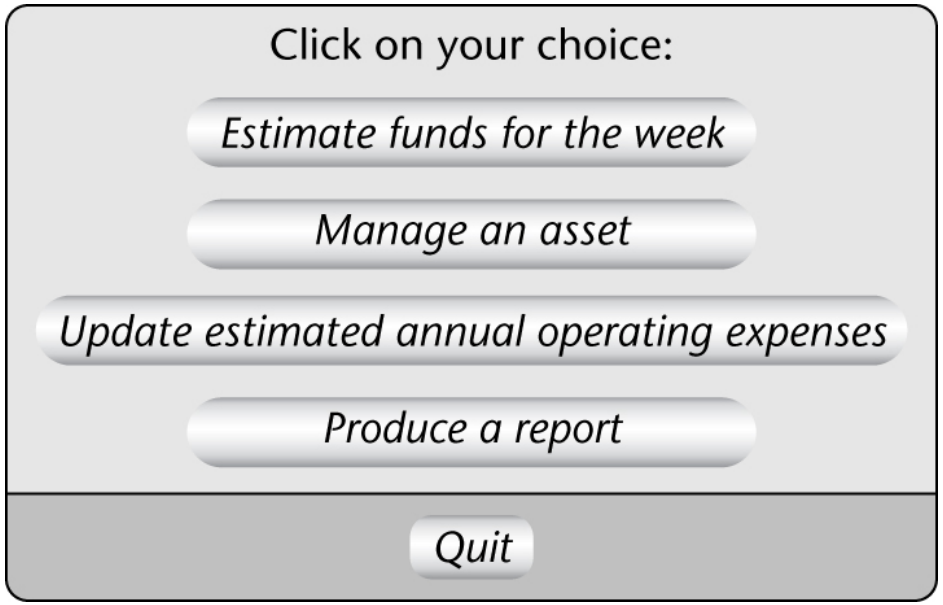
Initial Dynamic Model: MSG Foundation (contd)

- These possibilities are indicated by the five events
  - estimate funds for the week selected
  - manage an asset selected
  - update estimated annual operating expenses selected
  - produce a report selected, and
  - quit selected

- An event causes a transition between states

Initial Dynamic Model: MSG Foundation (contd)

- An MSG staff member selects an option by clicking on the menu



- This graphical user interface (GUI) requires special software

Initial Dynamic Model: MSG Foundation (contd)

- Equivalent textual user interface that can run on any computer

```
                    MAIN MENU
MARTHA STOCKTON GREENGAGE FOUNDATION
   1. Estimate funds available for week
   2. Manage an asset
   3. Update estimated annual operating expenses
   4. Produce a report
   5. Quit
Type your choice and press <ENTER>:
```

12.12 Revising the Entity Classes: MSG Foundation

- The initial functional model, the initial class diagram, and the initial dynamic model are completed
  - Checking them reveals a fault

- In the initial statechart, consider state **Update Estimated Annual Operating Expenses** with operation Update the estimated annual operating expenses
  - This operation has to be performed on the current value of the estimated annual operating expense

Revising the Entity Classes: MSG Foundation (contd)

- But where is the value of the estimated annual operating expenses to be found?
- Currently there is only one class (**Asset Class**) and its two subclasses
  - Neither is appropriate for storing the estimated annual operating expenses

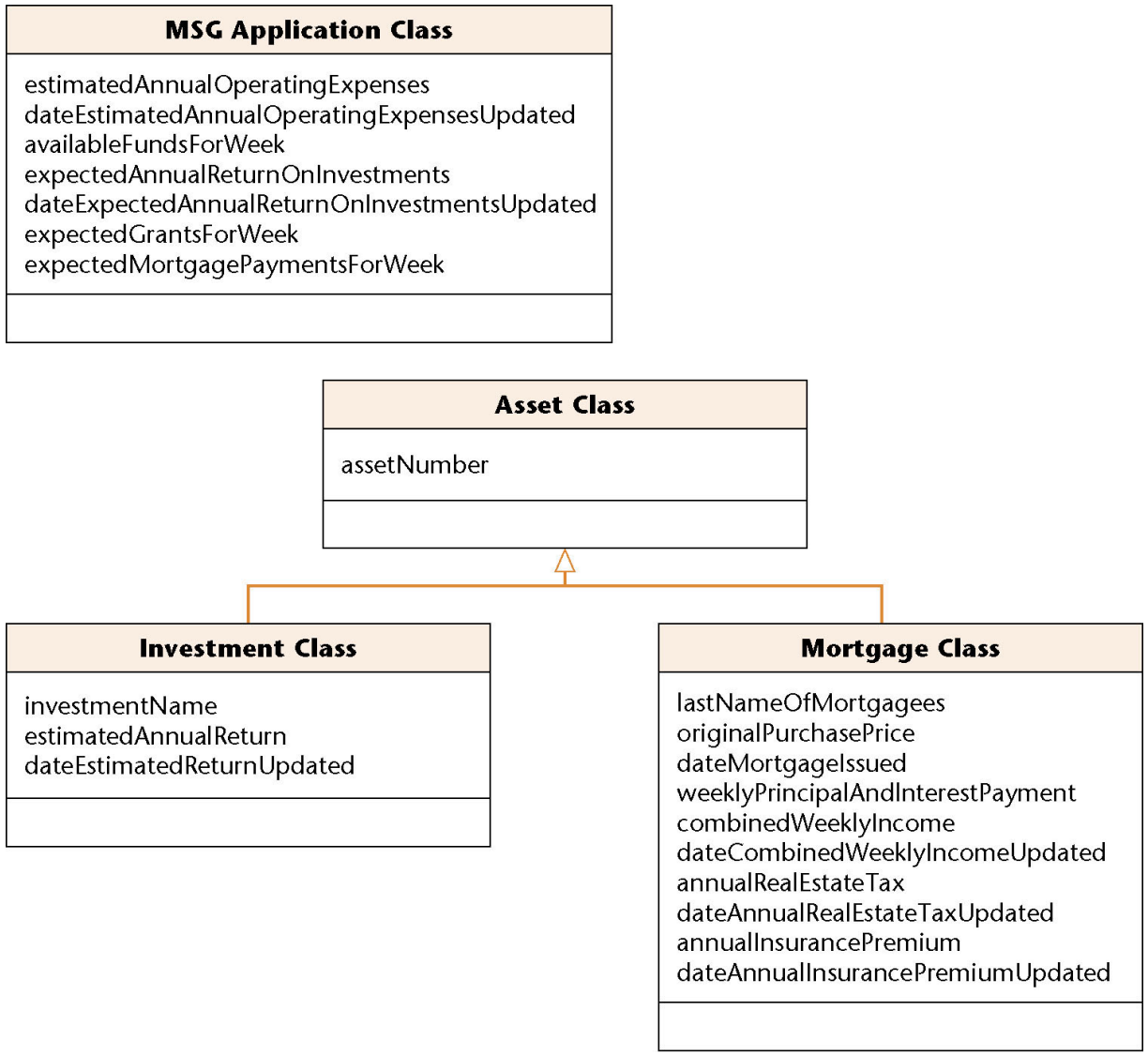
Revising the Entity Classes: MSG Foundation (contd)

- The only way a value can be stored on a long-term basis is as an attribute of an instance of that class or its subclasses

- Another entity class is needed for storing the estimated annual operating expenses
  - **MSG Application Class**

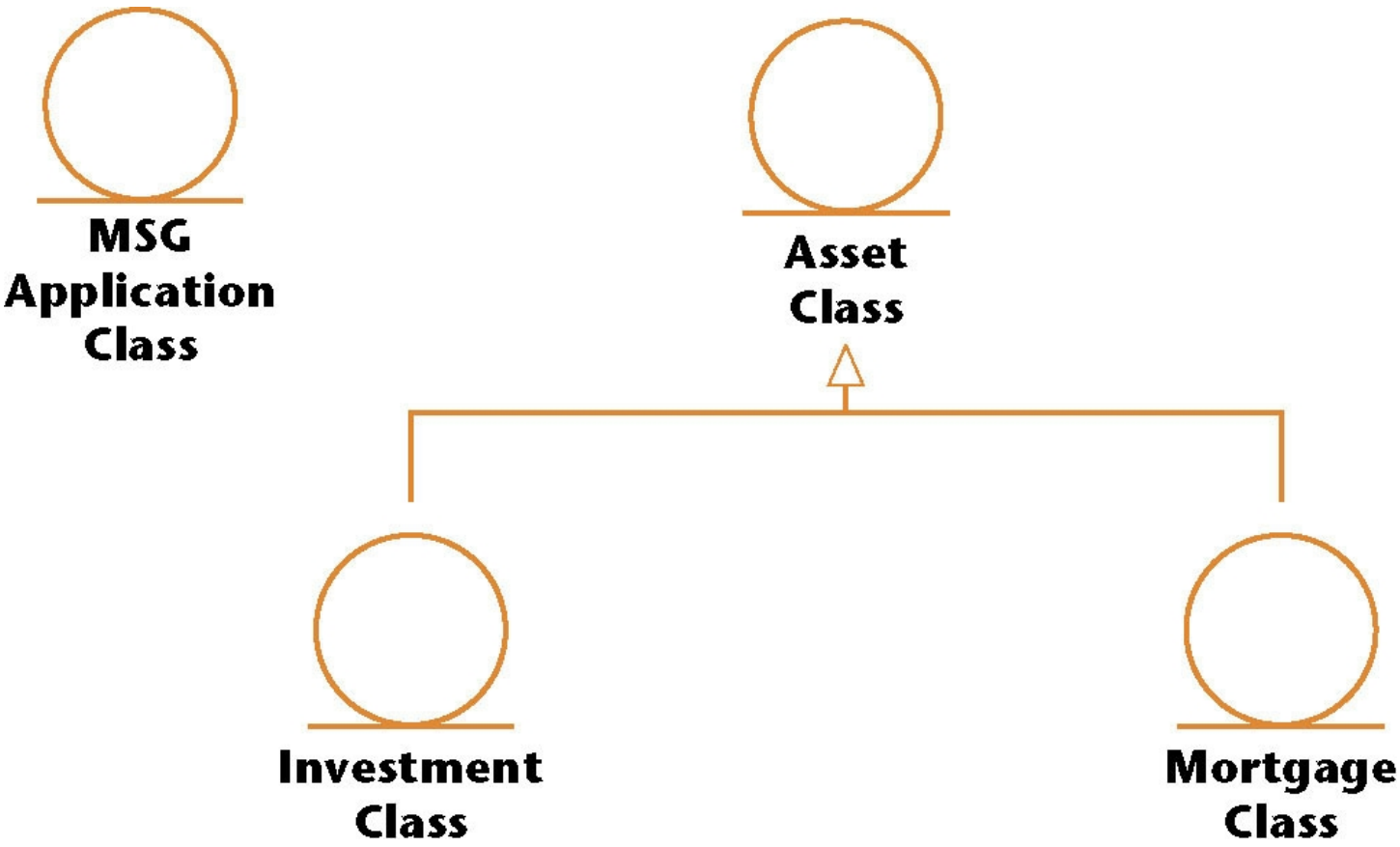
Third Iteration of the Initial Class Diagram: MSG Foundation

- **MSG Application Class** has other attributes as well
  - Attributes that do not appertain to the assets



Third Iteration of the Initial Class Diagram: MSG Foundation

1



12.13 Extracting the Boundary Classes: MSG Foundation

- It is usually easy to extract boundary classes
  - Each input screen, output screen, and printed report is generally modeled by a boundary class
- One screen should be adequate for all four MSG Foundation use cases
  - `Estimate Funds Available for Week`
  - `Manage an Asset`
  - `Update Estimated Annual Operating Expenses`
  - `Produce a Report`
- Accordingly there is one initial boundary class
  - **User Interface Class**

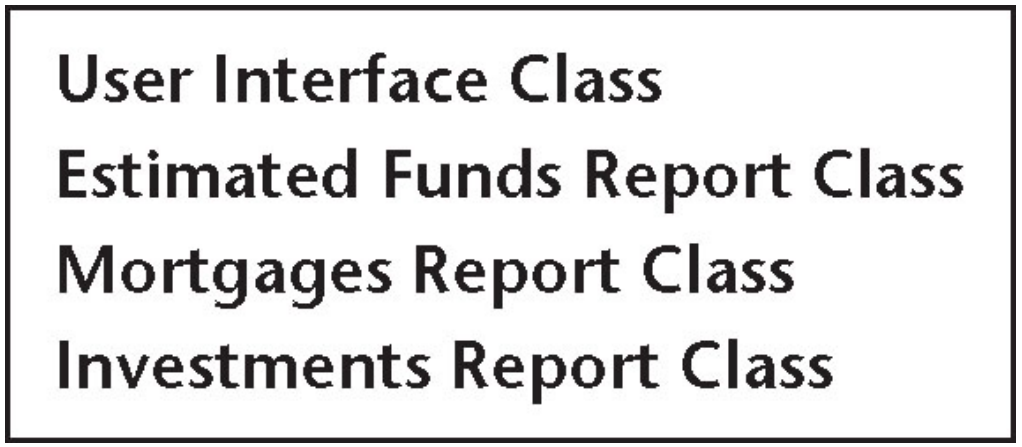
Extracting Boundary Classes: MSG Foundation (contd)

- Three reports have to be printed
  - The estimated funds for the week report
  - The listing of all mortgages
  - The listing of all investments

- Each of these has to be modeled by a separate boundary class
  - **Estimated Funds Report Class**
  - **Mortgages Report Class**
  - **Investments Report Class**

Extracting Boundary Classes: MSG (contd)

- Here are the four initial boundary classes



Initial Boundary Classes: MSG Foundation (contd)

- There are three reports:
  - The purchases report
  - The sales report
  - The future trends report
- The content of each report is different
  - Each report therefore has to be modeled by a separate boundary class

12.14 Extracting the Control Classes: MSG Foundation

- Each computation is usually modeled by a control class
- The MSG Foundation case study has just one
  - Estimate the funds available for the week
- There is one initial control class

Estimate Funds for Week Class

Class Extraction (contd)

- The description of class extraction is complete
- We now therefore return to the Unified Process

12.15 Use-Case Realization: The MSG Foundation Case Study

- The process of extending and refining use cases is called *use-case realization*

Use-Case Realization (contd)

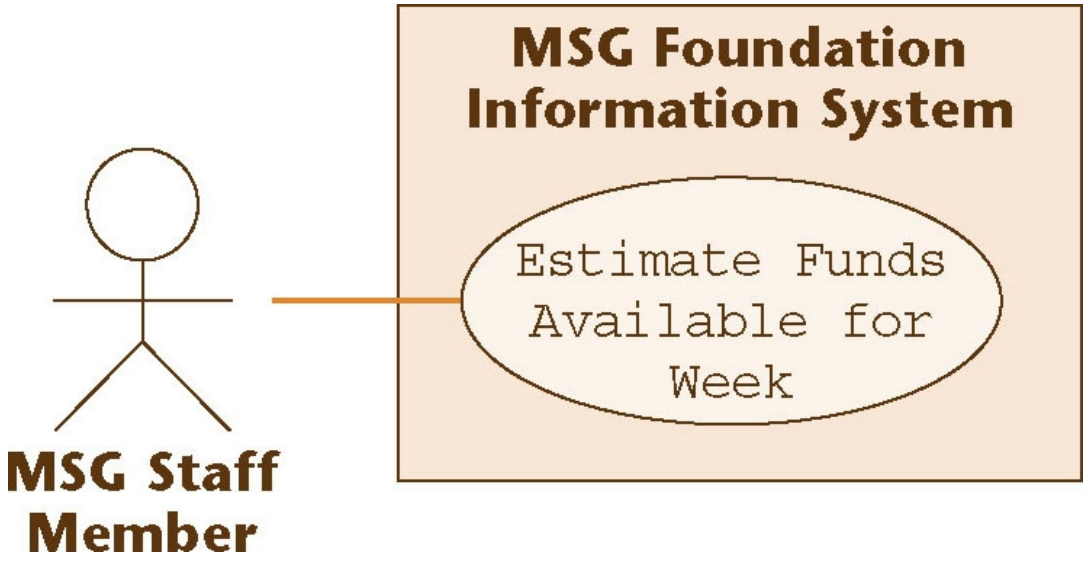
- The verb “realize” is used at least 3 different ways:
  - Understand (“Harvey slowly began to realize that he was in the wrong classroom”);
  - Receive (“Ingrid will realize a profit of $45,000 on the stock transaction”); and
  - Accomplish (“Janet hopes to realize her dream of starting a computer company”)
- In the phrase “realize a use case,” the word “realize” is used in this last sense

Use-Case Realization (contd)

- The realization of a specific scenario of a use case is depicted using an interaction diagram
  - Either a sequence diagram or collaboration diagram
- Consider use case `Estimate Funds Available for Week`
- We have previously seen
  - The use case
  - The description of the use case

12.15.1 Estimate Funds Available for Week Use Case

- Use-case diagram



Estimate Funds Available for Week Use Case (contd)

- Description of use case

Brief Description

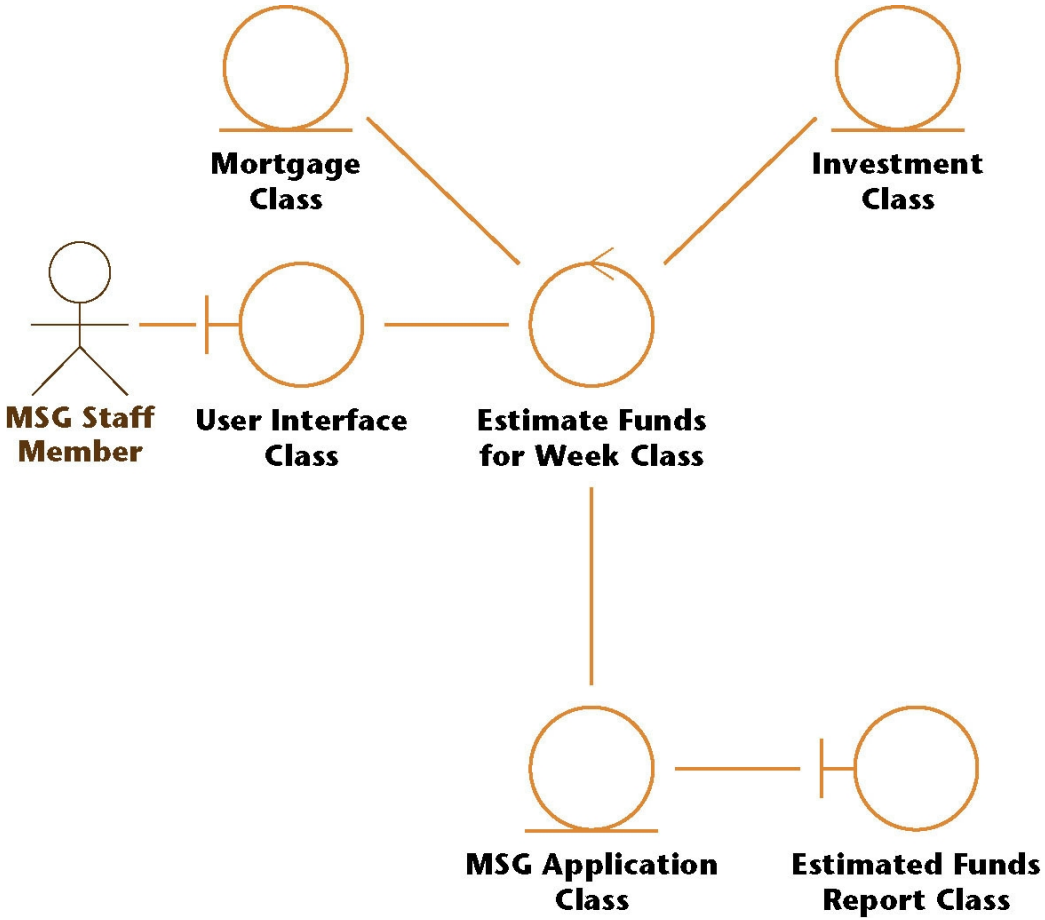
The Estimate Funds Available for Week use case enables an MSG Foundation staff member to estimate how much money the Foundation has available that week to fund mortgages.

Step-by-Step Description

1. For each investment, extract the estimated annual return on that investment. Summing the separate returns and dividing the result by 52 yields the estimated investment income for the week.
2. Determine the estimated MSG Foundation operating expenses for the week by extracting the estimated annual MSG Foundation operating expenses and dividing by 52.
3. For each mortgage:
   - 3.1 The amount to be paid this week is the total of the principal and interest payment and $\frac{1}{52}$nd of the sum of the annual real-estate tax and the annual homeowner's insurance premium.
   - 3.2 Compute 28 percent of the couple's current gross weekly income.
   - 3.3 If the result of Step 3.1 is greater than the result of Step 3.2, then the mortgage payment for this week is the result of Step 3.2, and the amount of the grant for this week is the difference between the result of Step 3.1 and the result of Step 3.2.
   - 3.4 Otherwise, the mortgage payment for this week is the result of Step 3.1, and there is no grant this week.
4. Summing the mortgage payments of Steps 3.3 and 3.4 yields the estimated total mortgage payments for the week.
5. Summing the grant payments of Step 3.3 yields the estimated total grant payments for the week.
6. Add the results of Steps 1 and 4 and subtract the results of Steps 2 and 5. This is the total amount available for mortgages for the current week.
7. Print the total amount available for new mortgages during the current week.

Estimate Funds Available for Week Use Case (contd)

- Class diagram (classes that enter into the use case)



Estimate Funds Available for Week Use Case (contd)

- The six classes that enter into this use case are:
  - **User Interface Class**
    - This class models the user interface
  - **Estimate Funds for Week Class**
    - This control class models the computation of the estimate of the funds that are available to fund mortgages during that week
  - **Mortgage Class**
    - This class models the estimated grants and payments for the week
  - **Investment Class**
    - This class models the estimated return on investments for the week
  - **MSG Application Class**
    - This class models the estimated return on investments for the week
  - **Estimated Funds Report Class**

Estimate Funds Available for Week Use Case (contd)

- Scenario (one possible instance of the use case)

An MSG Foundation staff member wishes to determine the funds available for mortgages this week.

1. For each investment, the information system extracts the estimated annual return on that investment. It sums the separate returns and divides the result by 52 to yield the estimated investment income for the week.
2. The information system then extracts the estimated annual MSG Foundation operating expenses and divides the result by 52.
3. For each mortgage:
   - 3.1 The information system computes the amount to be paid this week by adding the principal and interest payment to $\frac{1}{52}$nd of the sum of the annual real-estate tax and the annual homeowner's insurance premium.
   - 3.2 It then computes 28 percent of the couple's current gross weekly income.
   - 3.3 If the result of Step 3.1 is greater than the result of Step 3.2, then it determines the mortgage payment for the week as the result of Step 3.2, and the amount of the grant for this week as the difference between the result of Step 3.1 and the result of Step 3.2.
   - 3.4 Otherwise, it takes the mortgage payment for this week as the result of Step 3.1, and there is no grant for the week.
4. The information system sums the mortgage payments of Steps 3.3 and 3.4 to yield the estimated total mortgage payments for the week.
5. It sums the grant payments of Step 3.3 to yield the estimated total grant payments for the week.
6. The information system adds the results of Steps 1 and 4 and subtracts the results of Steps 2 and 5. This is the total amount available for mortgages for the current week.
7. Finally, the software product prints the total amount available for new mortgages during the current week.

`Estimate Funds Available for Week` Use Case (contd)

- A working information system uses objects, not classes
  - Example: A specific mortgage cannot be represented by **Mortgage Class** but rather by an object, a specific instance of **Mortgage Class**

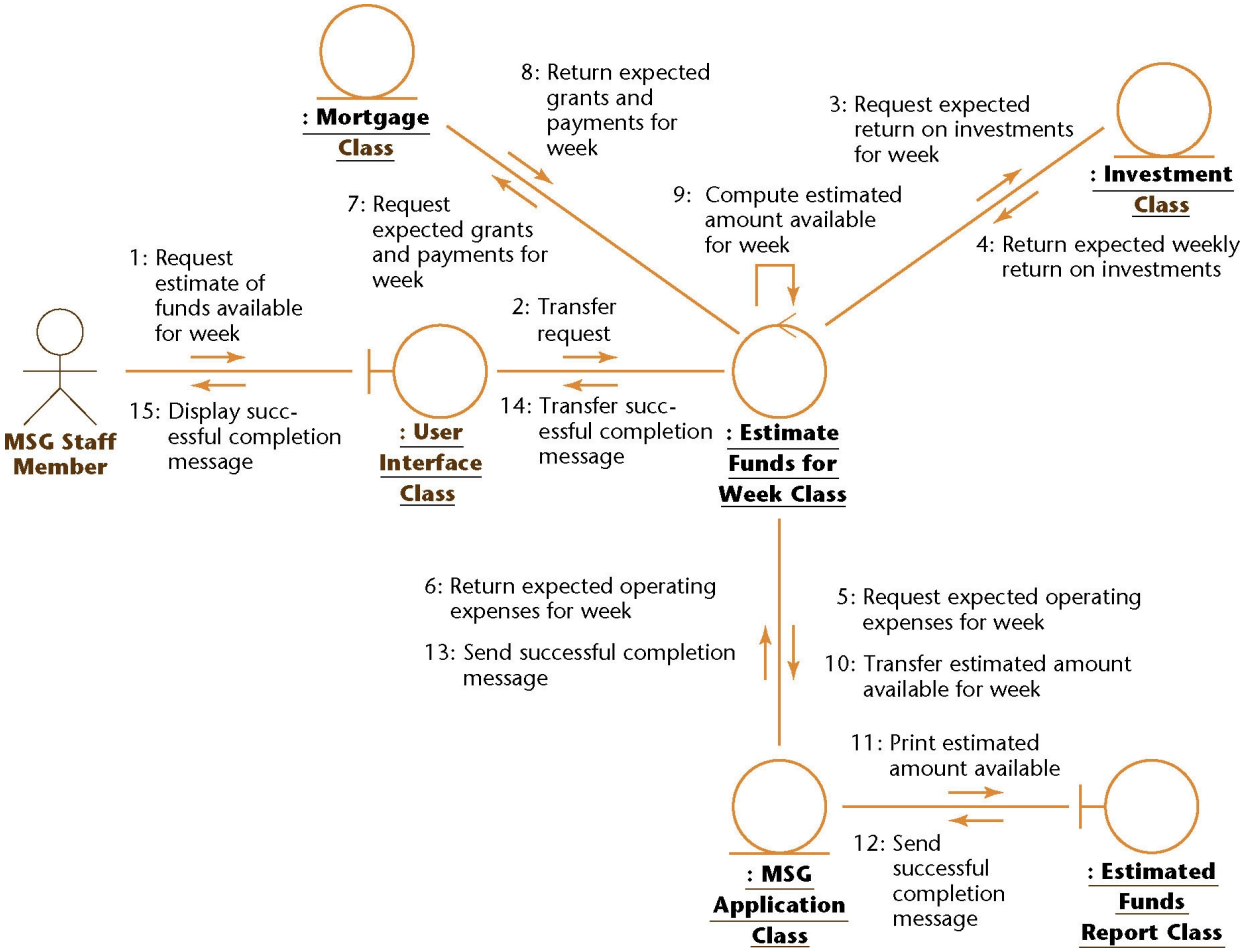
- Such an object is denoted by **: Mortgage Class**

`Estimate Funds Available for Week` Use Case (contd)

- A class diagram shows the classes in the use case and their relationships
  - It does not show the objects nor the sequence of messages as they are sent from object to object
- Something more is needed

Estimate Funds Available for Week Use Case (contd)

- Collaboration diagram (of the realization of the scenario of the use case)



`Estimate Funds Available for Week` Use Case (contd)

- The collaboration diagram shows the objects as well as the messages, numbered in the order in which they are sent in the specific scenario

Estimate Funds Available for Week Use Case (contd)

- Item 1:
  - The staff member wants to compute the funds available for the week
  - In the collaboration diagram, this is modeled by message
    - 1: Request estimate of funds available for week

    from **MSG Staff Member** to **: User Interface Class**, an instance of **User Interface Class**

Estimate Funds Available for Week Use Case (contd)

- Item 2
  - This request is passed on to **: Estimate Funds for Week Class**, an instance of the control class that actually performs the calculation
  - This is modeled by message
    - 2: Transfer request

- Four separate financial estimates are now determined by **: Estimate Funds for Week Class**

- Item 3
  - In Step 1 of the scenario, the estimated annual return on investments is summed for each investment and the result divided by 52
  - This extraction of the estimated weekly return is modeled by message
    - 3: Request estimated return on investments for week

    from **: Estimate Funds for Week Class**
    to **: Investment Class** followed by message
    - 4: Return estimated weekly return on investments

    in the other direction

- Item 4
  - In Step 2 of the scenario, the weekly operating expenses are estimated by taking the estimated annual operating expenses and dividing by 52
  - This extraction of the weekly expenses is modeled by message
    - 5: Request estimated operating expenses for week

    from **: Estimate Funds for Week Class** to **: MSG Application Class** followed by message
    - 6: Return estimated operating expenses for week

    in the other direction

- Item 5
  - In Steps 3, 4, and 5 of the scenario, two estimates are determined
    - the estimated grants for the week, and
    - the estimated payments for the week
  - This is modeled by message
    - 7: Request estimated grants and payments for week

    from **: Estimate Funds for Week Class** to **: Mortgage Class**, and by message
    - 8: Return estimated grants and payments for week

    in the other direction

- Item 6
  - Now the arithmetic computation of Step 6 of the scenario is performed
  - This is modeled by message
    - 9: Compute estimated amount available for week
  - This is a self call
  - **: Estimate Funds for Week Class** tells itself to perform the calculation
  - The result of the computation is stored in **: MSG Application Class** by message
    - 10: Transfer estimated amount available for week

- Item 7
  - The result is printed in Step 7 of the scenario
  - This is modeled by message
    - 11: Print estimated amount available
  - from **: MSG Application Class** to **: Estimated Funds Report Class**

- Item 8
  - Finally, an acknowledgment is sent to the MSG staff member that the task has been successfully completed
  - This is modeled by messages
    - 12: Send successful completion message
    - 13: Send successful completion message
    - 14: Transfer successful completion message, and
    - 15: Display successful completion message

`Estimate Funds Available for Week` Use Case (contd)

- No client will approve the specification document without understanding it
- Accordingly, a written description of the collaboration diagram is needed, the flow of events

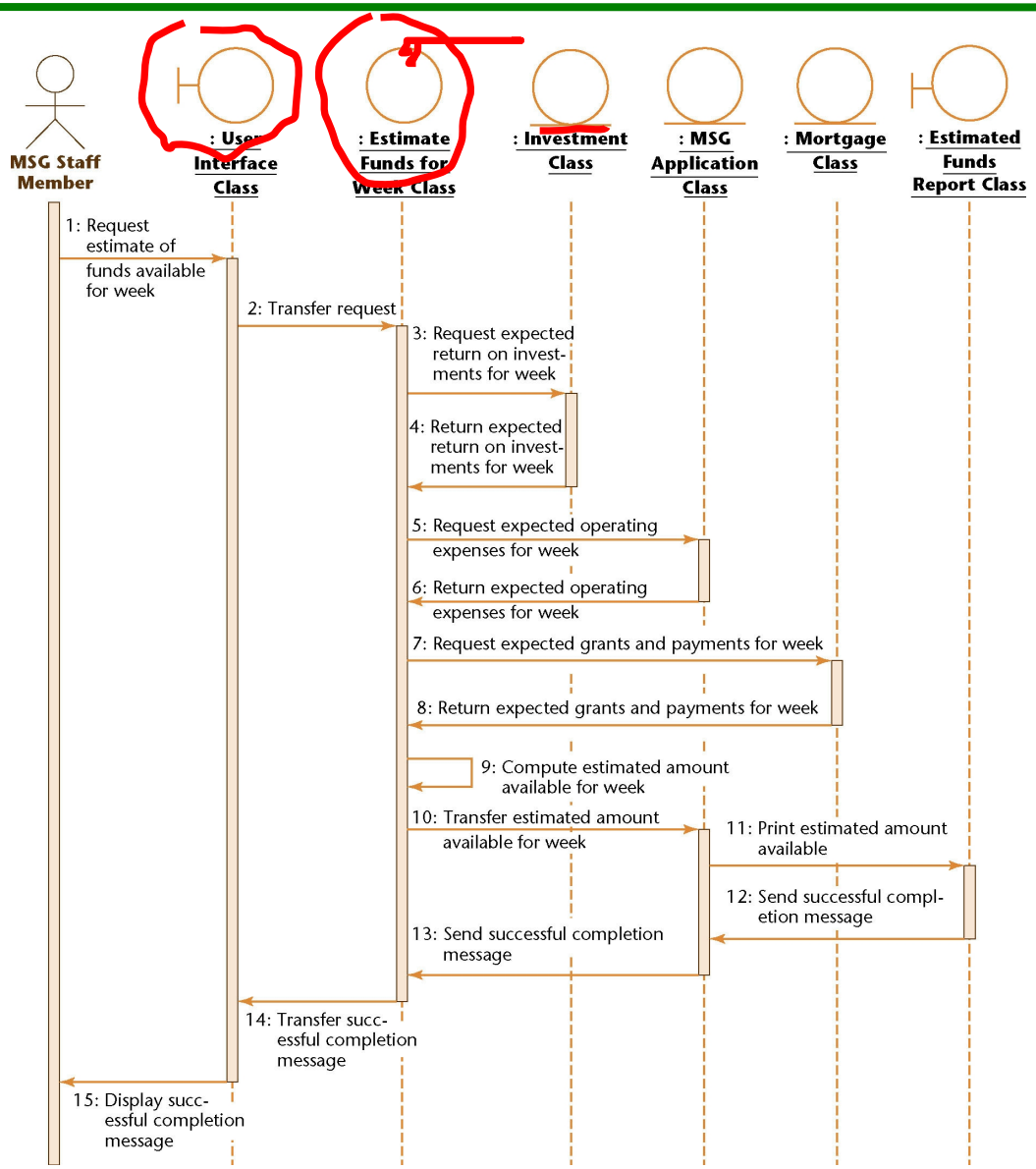
`Estimate Funds Available for Week` Use Case (contd)

- The flow of events of the collaboration diagram of the realization of the scenario of the use case

An MSG staff member requests an estimate of the funds available for mortgages for the week (1, 2). The information system estimates the return on investments for the week (3, 4), the operating expenses for the week (5, 6), and the grants and payments for the week (7, 8). Then it estimates (9), stores (10), and prints out (11–15) the funds available for the week.

Estimate Funds Available for Week Use Case (contd)

- Sequence diagram equivalent to the collaboration diagram (of the realization of the scenario of the use case)



Interaction Diagrams

- The strength of a sequence diagram is that it shows the flow of messages and their order unambiguously
  - When transfer of information is the focus of attention, a sequence diagram is superior to a collaboration diagram
- A collaboration diagram is similar to a class diagram
  - When the developers are concentrating on the classes, a collaboration diagram is more useful than the equivalent sequence diagram

`Estimate Funds Available for Week` Use Case (contd)

- Figures 12.29 through 12.35 do not depict a random collection of UML artifacts
- Instead, these figures depict a use case and artifacts derived from that use case
- In more detail (see next slide):

`Estimate Funds Available for Week` Use Case (contd)

- Figure 12.29 depicts the use case `Estimate Funds Available for Week`

- The figure models
  - All possible sets of interactions
  - Between the actor **MSG Staff Member** (external to the software product) and the MSG Foundation software product itself
  - That relate to the action of estimating funds available for the week

`Estimate Funds Available for Week` Use Case (contd)

- Figure 12.30 is the description of that use case

- The figure provides a written account of the details of the `Estimate Funds Available for Week` use case of Figure 12.29

`Estimate Funds Available for Week` Use Case (contd)

- Figure 12.31 is a class diagram showing the classes that realize the `Estimate Funds Available for Week` use case

- The figure depicts
  - The classes that are needed to model all possible scenarios of the use case
  - Together with their interactions

`Estimate Funds Available for Week` Use Case (contd)

- Figure 12.32 is a scenario
- It depicts one specific instance of the use case of Figure 12.29

`Estimate Funds Available for Week` Use Case (contd)

- Figure 12.33 is a collaboration diagram of the realization of the scenario of Figure 12.32

- The figure depicts the objects and the messages sent between them in the realization of that one specific scenario

`Estimate Funds Available for Week` Use Case (contd)

- Figure 12.34 is the flow of events of the collaboration diagram of the realization of the scenario of Figure 12.32

- Figure 12.34 is a written description of the realization of the scenario of Figure 12.32

  - (Compare: Figure 12.30 is a written description of the `Estimate Funds Available for Week` use case of Figure 12.29)

Estimate Funds Available for Week Use Case (contd)

- Figure 12.35 is the sequence diagram that is fully equivalent to the collaboration diagram of Figure 12.33
- The sequence diagram depicts the objects and the messages sent between them in the realization of the scenario of Figure 12.32
- Its flow of events is therefore also shown in Figure 12.34

12.5.2 Manage an Asset Use Case

- Use case



Manage an Asset Use Case (contd)

- Description of use case

Brief Description

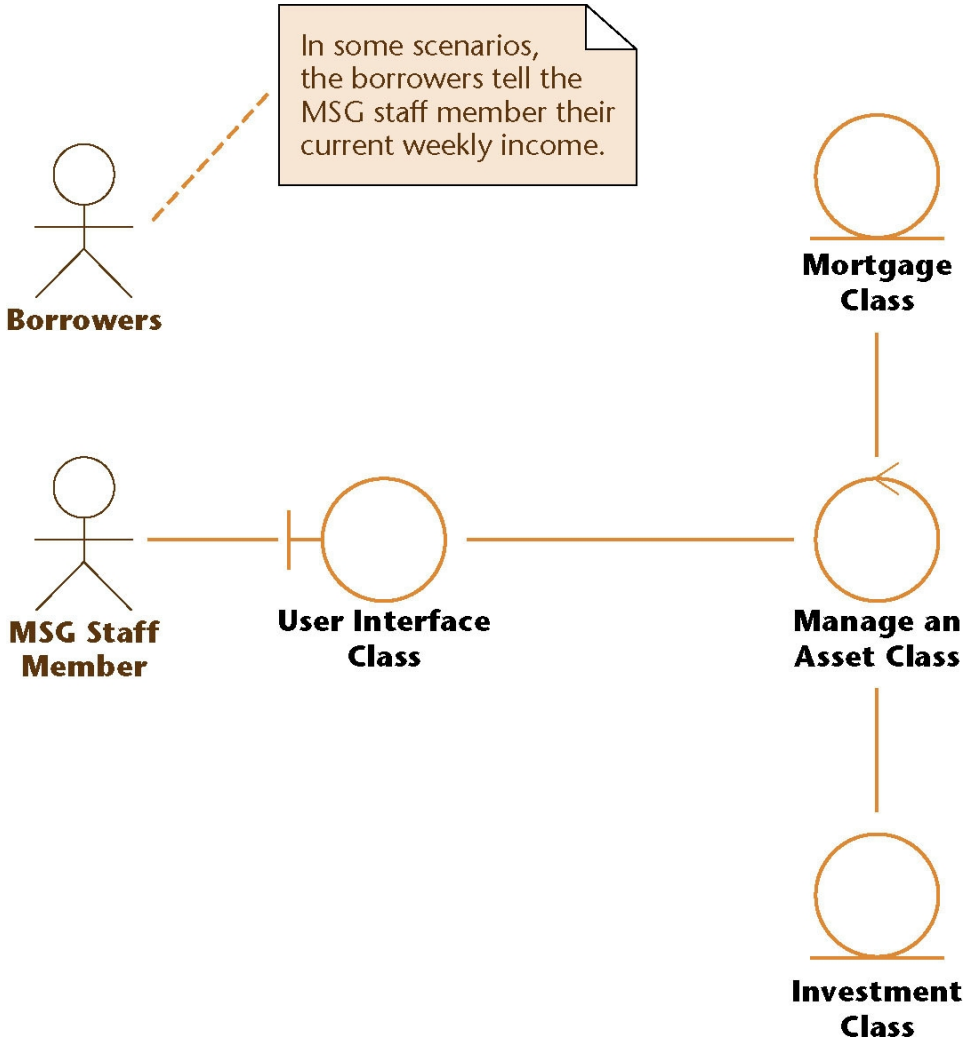
The Manage an Asset use case enables an MSG Foundation staff member to add and delete assets and manage the portfolio of assets (investments and mortgages). Managing a mortgage includes updating the weekly income of a couple who have borrowed money from the Foundation.

Step-by-Step Description

1. Add, modify, or delete an investment or mortgage, or update the borrower's weekly income.

Manage an Asset Use Case (contd)

- Class diagram showing the classes that realize the use case



Manage an Asset Use Case (contd)

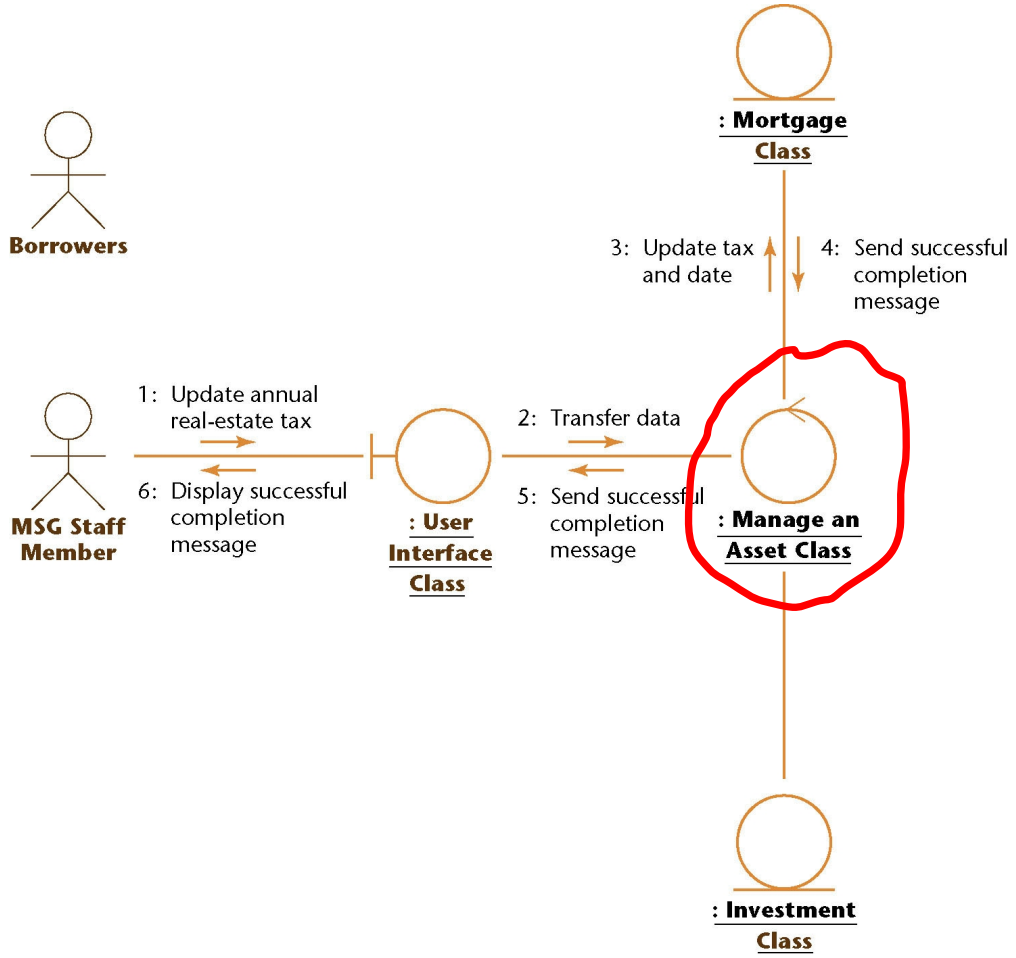
- One scenario of the use case

An MSG Foundation staff member wants to update the annual real-estate tax on a home for which the Foundation has provided of a mortgage.

1. The staff member enters the new value of the annual real-estate tax.
2. The information system updates the date on which the annual real-estate tax was last changed.

Manage an Asset Use Case (contd)

- Collaboration diagram of the realization of the scenario of the use case

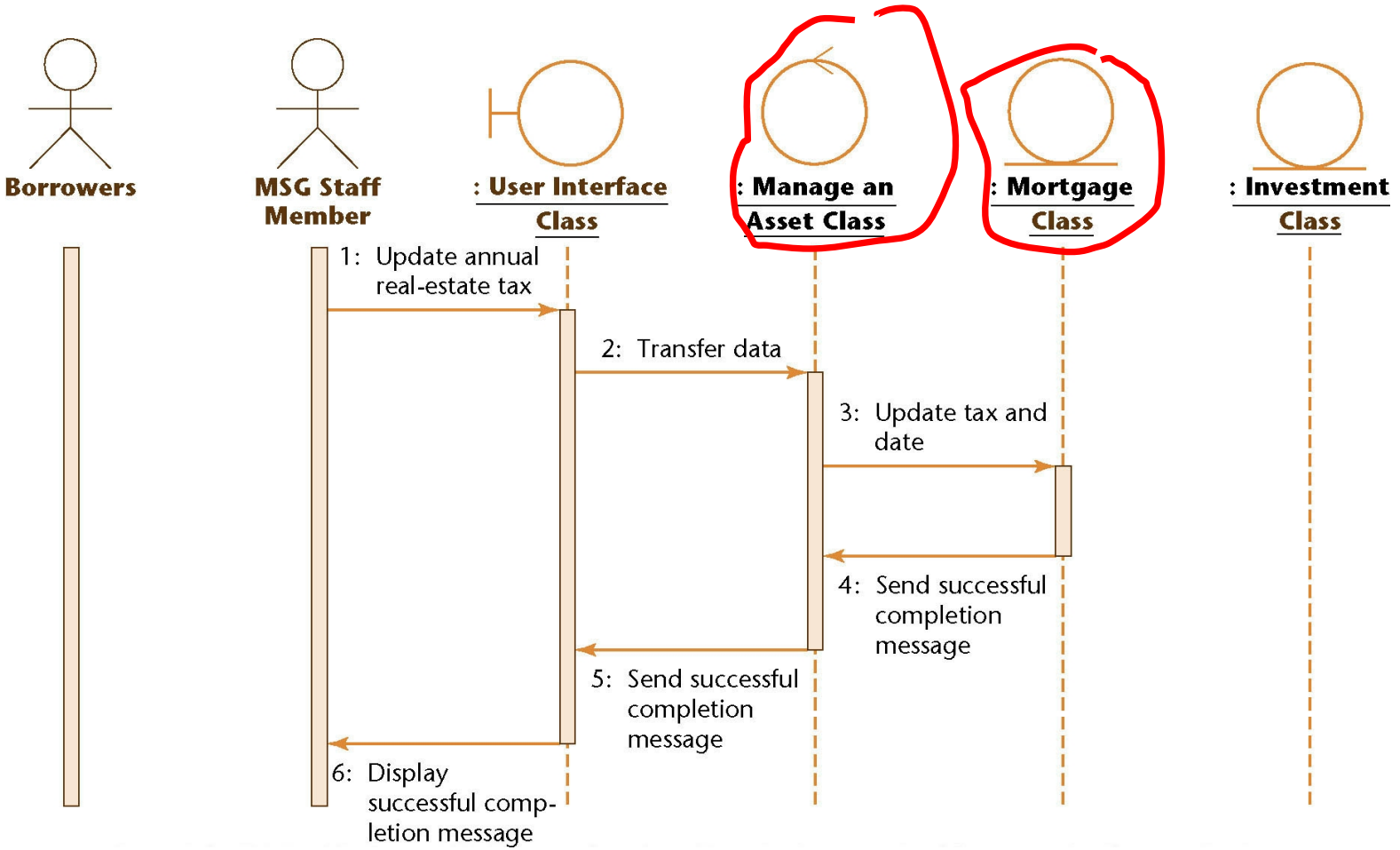


Manage an Asset Use Case (contd)

- Object **: Investment Class** does not play an active role in this collaboration diagram
  - This scenario does not involve an investment, only a mortgage
- Actor **Borrowers** does not play a role in this use case, either

Manage an Asset Use Case (contd)

1



Manage an Asset Use Case (contd)

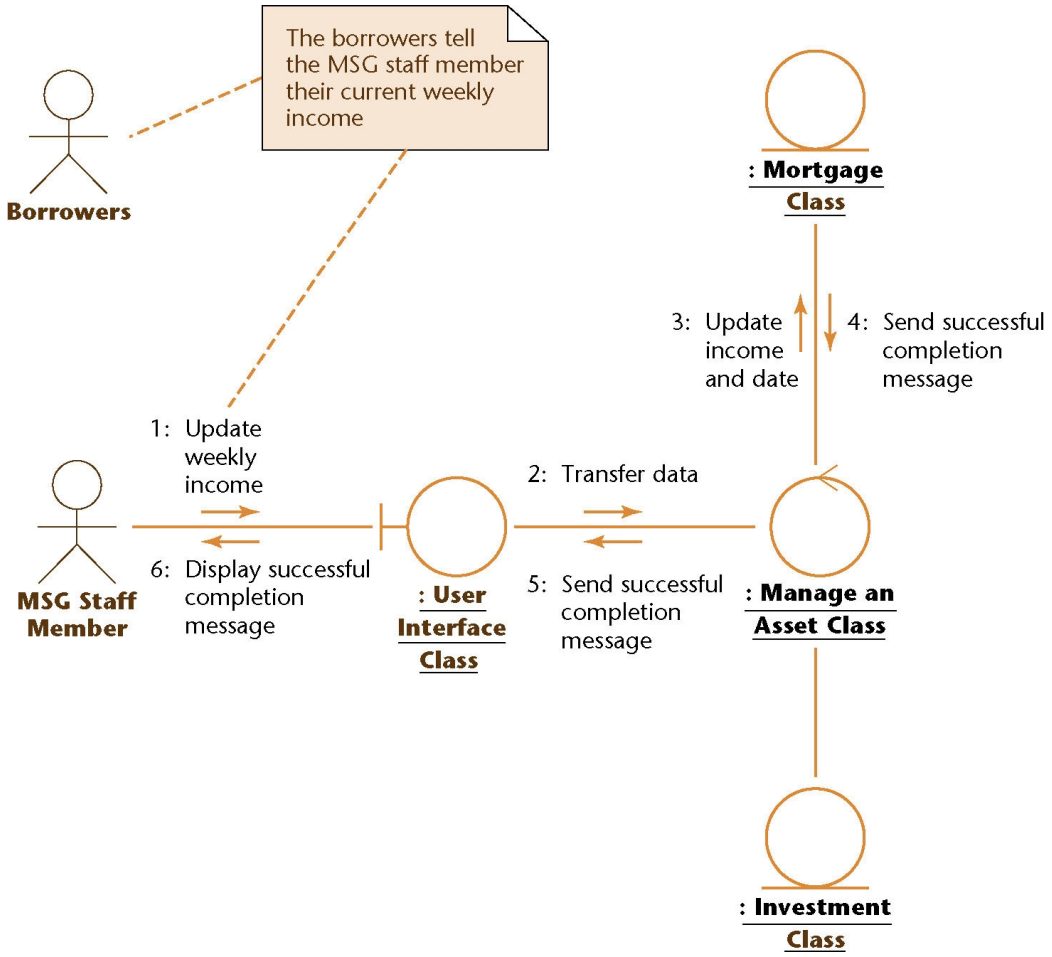
- A different scenario of the use case

There is a change in the weekly income of a couple who have borrowed money from the MSG Foundation. They wish to have their weekly income updated in the Foundation records by an MSG staff member so that their mortgage payments will be correctly computed.

1. The staff member enters the new value of the weekly income.
2. The information system updates the date on which the weekly income was last changed.

Manage an Asset Use Case (contd)

- Collaboration diagram of the realization of the scenario of the use case

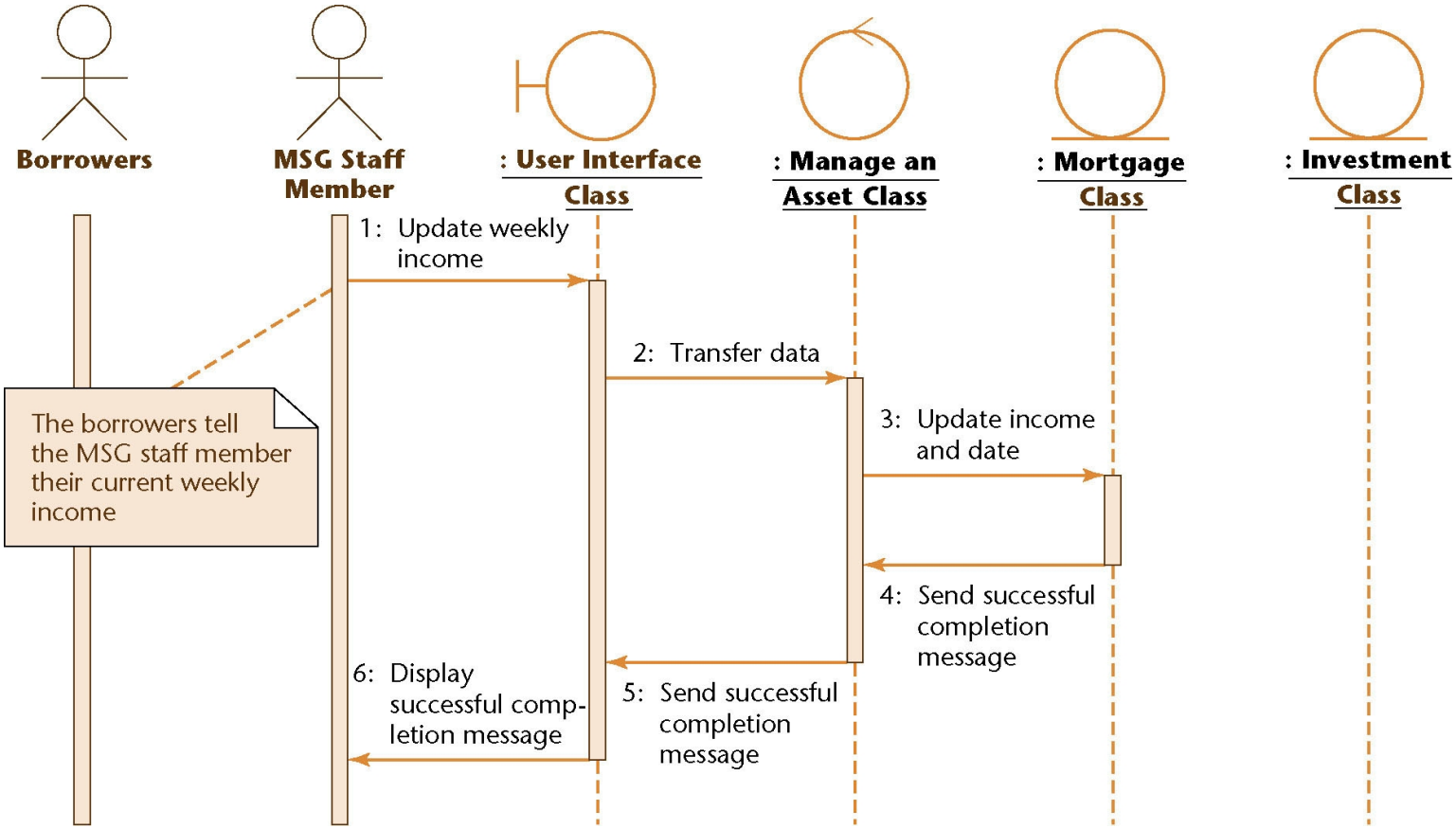


Manage an Asset Use Case (contd)

- At the request of the borrowers, the MSG staff member updates the weekly income of a couple
- The scenario is initiated by the **Borrowers**
- Their data are entered into the software product by the **MSG Staff Member**
  - This is stated in the note in the collaboration diagram

Manage an Asset Use Case (contd)

- Sequence diagram equivalent to the collaboration diagram (of the realization of the scenario of the use case)

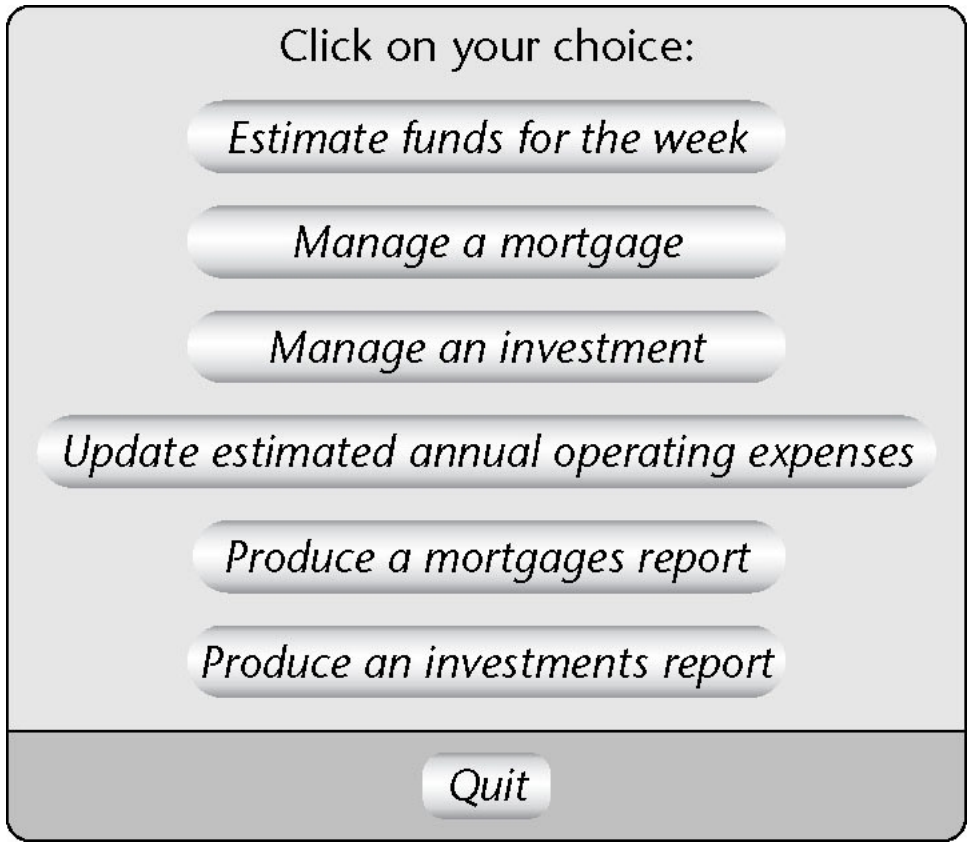


Manage an Asset Use Case (contd)

- Two different scenarios of the same use case have been presented
- The use case is the same
  - The class diagram is therefore the same
- However, the collaboration (and sequence) diagrams reflect the differences between the two scenarios

Manage an Asset Use Case (contd)

- Boundary class **User Interface Class** appears in all the realizations
  - The same screen will be used for all commands of the information system
- Revised menu



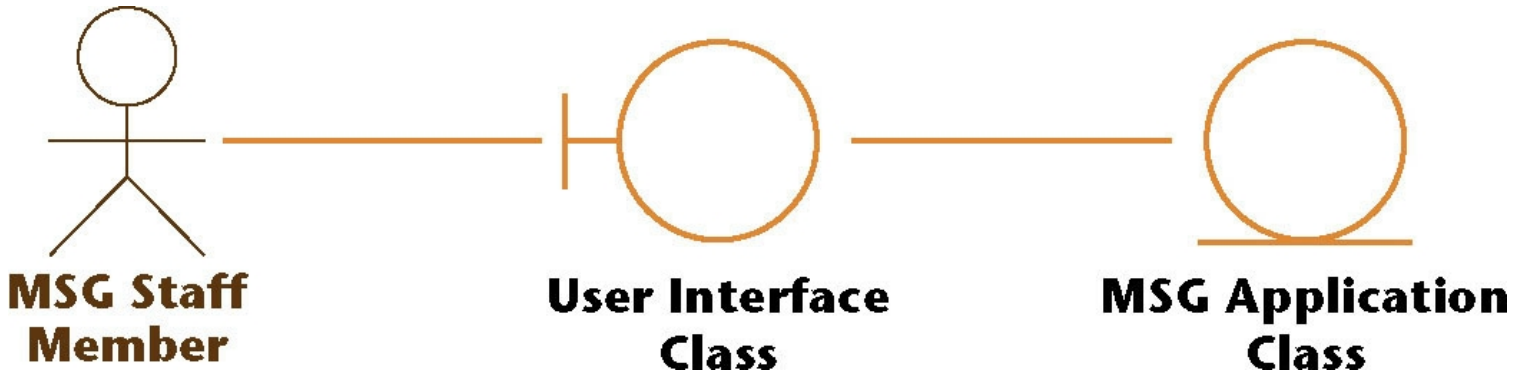
Manage an Asset Use Case (contd)

- Corresponding textual interface

```
                    MAIN MENU
MARTHA STOCKTON GREENGAGE FOUNDATION
  1. Estimate funds available for week
  2. Manage a mortgage
  3. Manage an investment
  4. Update estimated annual operating expenses
  5. Produce a mortgages report
  6. Produce an investments report
  7. Quit
 Type your choice and press <ENTER>:
```

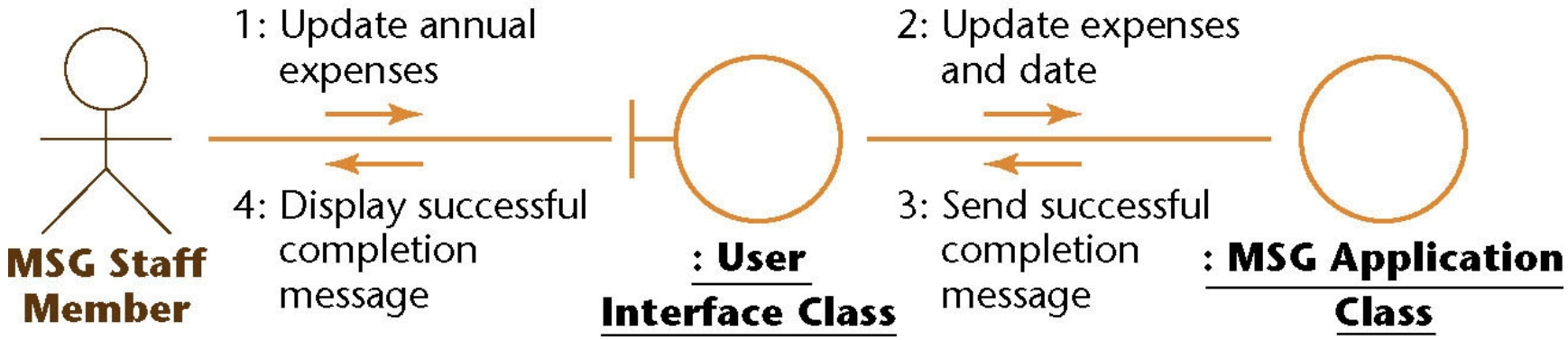
Update Annual Operating Expenses Use Case

- Class diagram



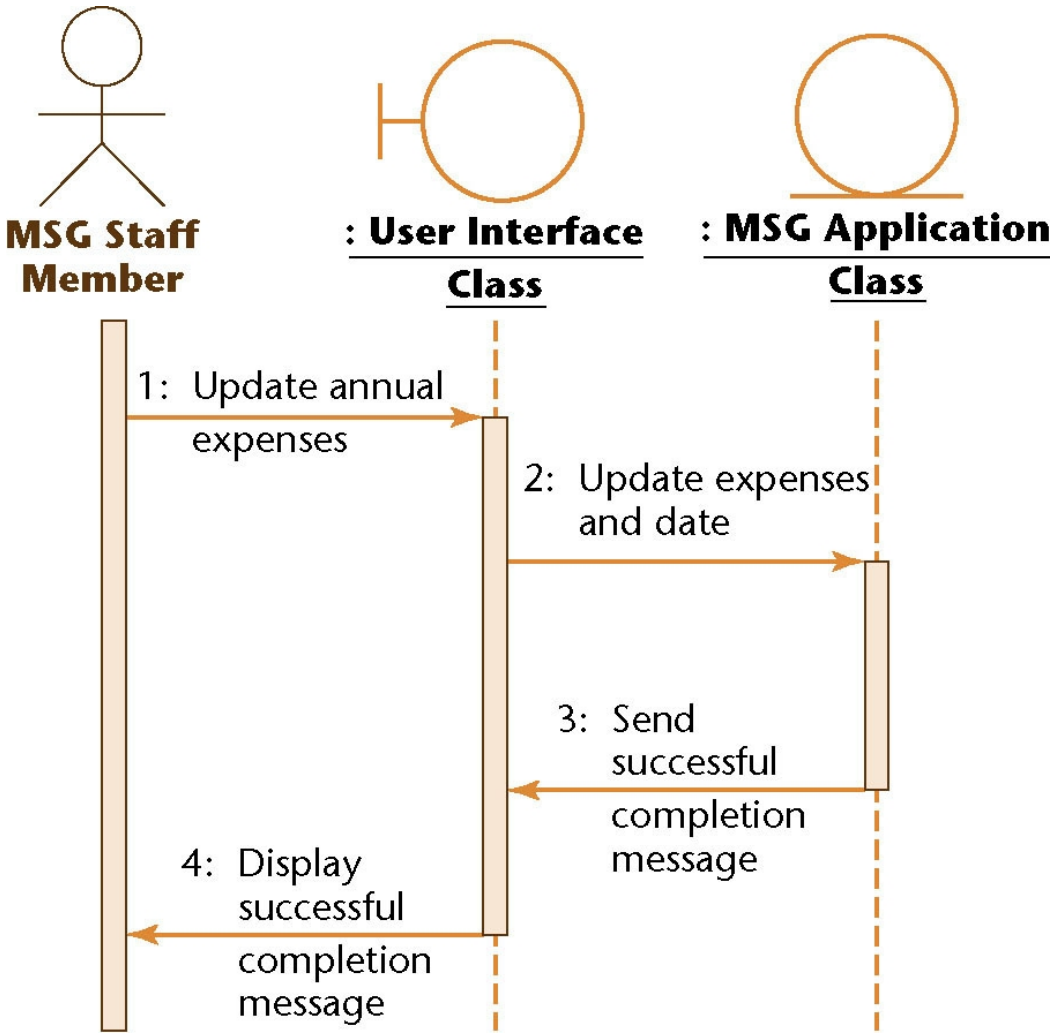
Update Annual Operating Expenses Use Case (contd)

- Collaboration diagram of a realization of a scenario of the use case



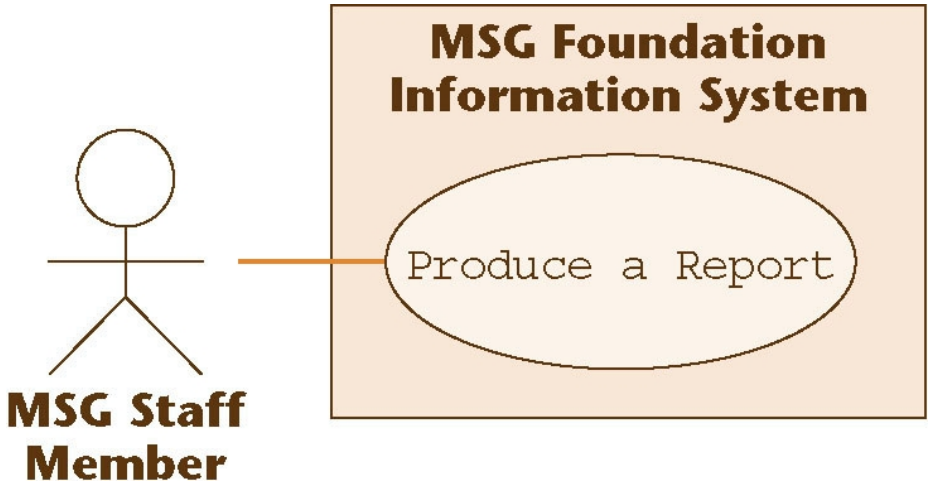
Update Annual Operating Expenses Use Case (contd)

- Equivalent sequence diagram



12.15.4 Produce a Report Use Case

- Use case



Produce a Report Use Case (contd)

- Description of use case

Brief Description

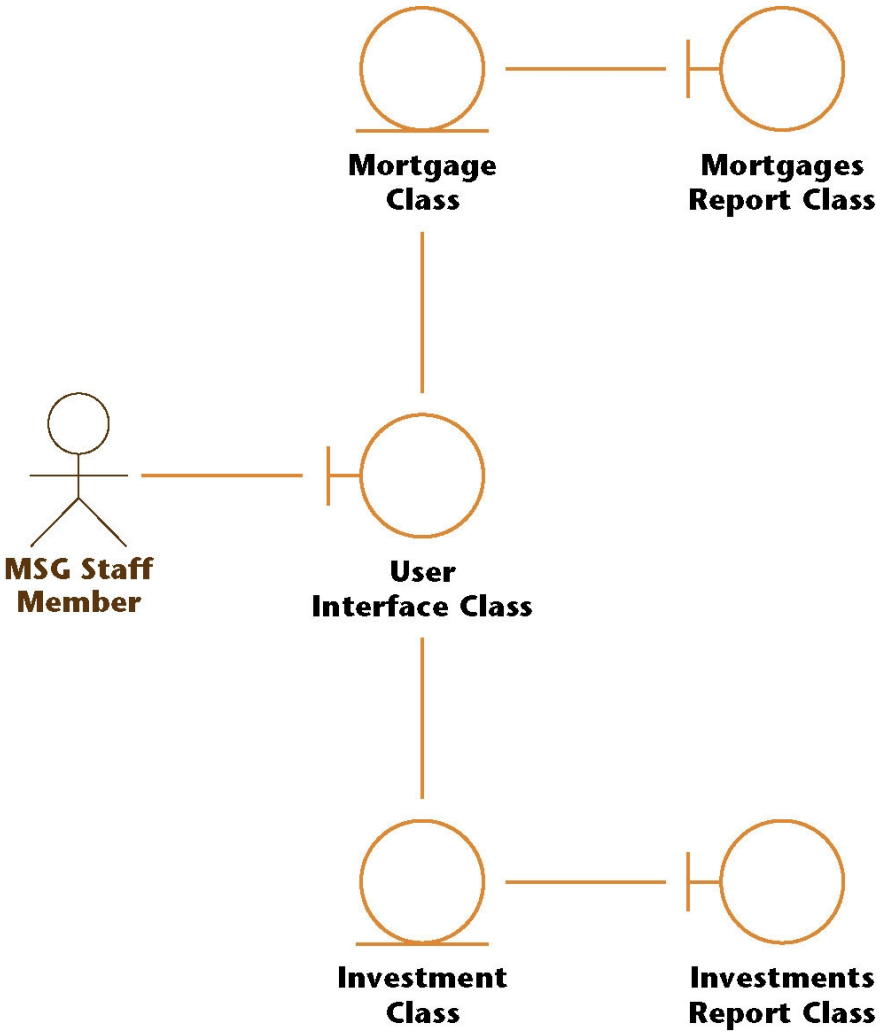
The Produce a Report use case enables an MSG Foundation staff member to print a listing of all investments or all mortgages.

Step-by-Step Description

1. The following reports must be generated:
   1.1 Investments report—printed on demand:
   The information system prints a list of all investments. For each investment, the following attributes are printed:
   Item number
   Item name
   Estimated annual return
   Date estimated annual return was last updated
   1.2 Mortgages report—printed on demand:
   The information system prints a list of all mortgages. For each mortgage, the following attributes are printed:
   Account number
   Name of mortgagees
   Original price of home
   Date mortgage was issued
   Principal and interest payment
   Current combined gross weekly income
   Date current combined gross weekly income was last updated
   Annual real-estate tax
   Date annual real-estate tax was last updated
   Annual homeowner's insurance premium
   Date annual homeowner's insurance premium was last updated

Produce a Report Use Case (contd)

- Class diagram



`Produce a Report` Use Case (contd)

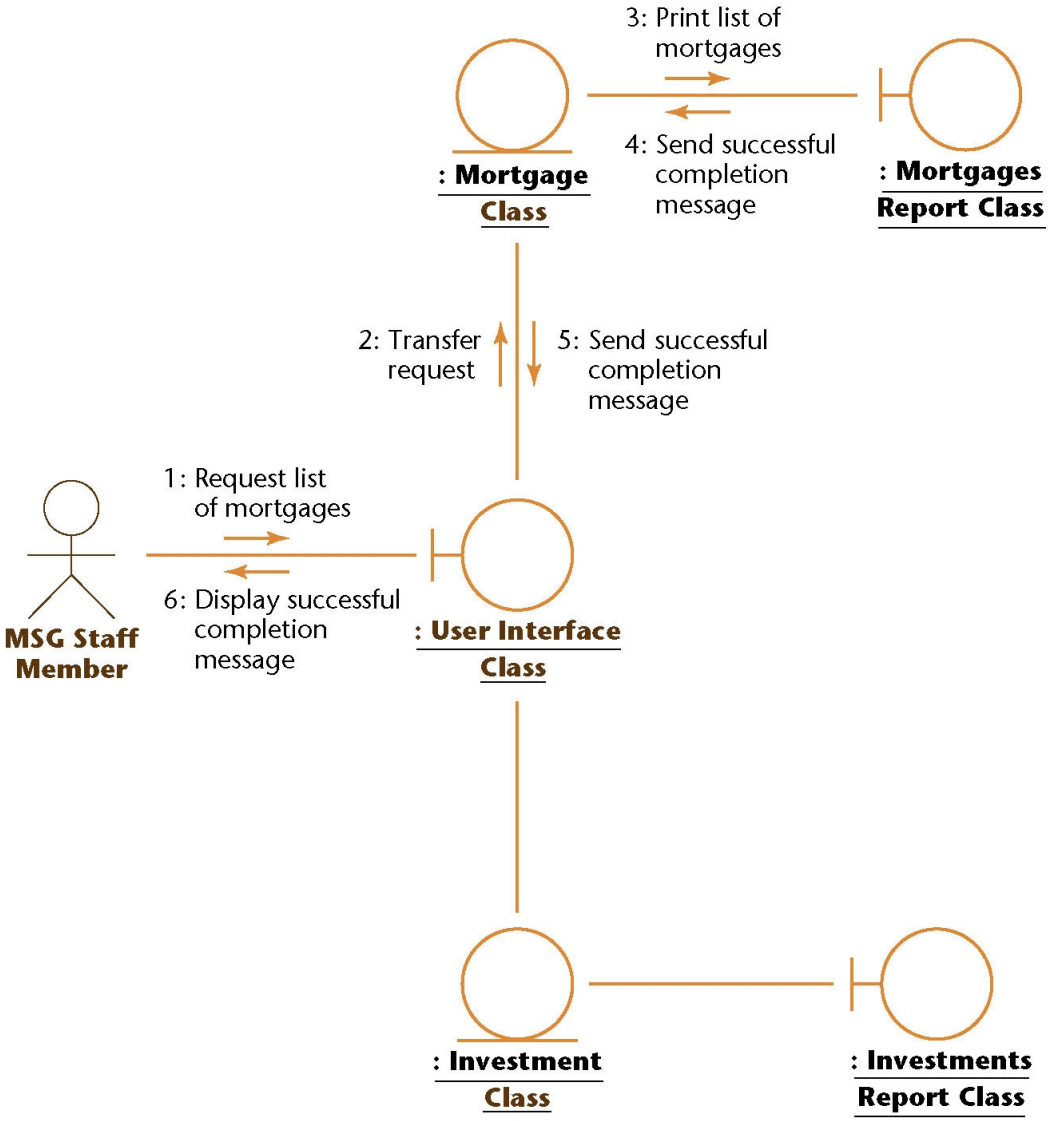
- One scenario of the use case

An MSG staff member wishes to print a list of all mortgages.

1. The staff member requests a report listing all mortgages.

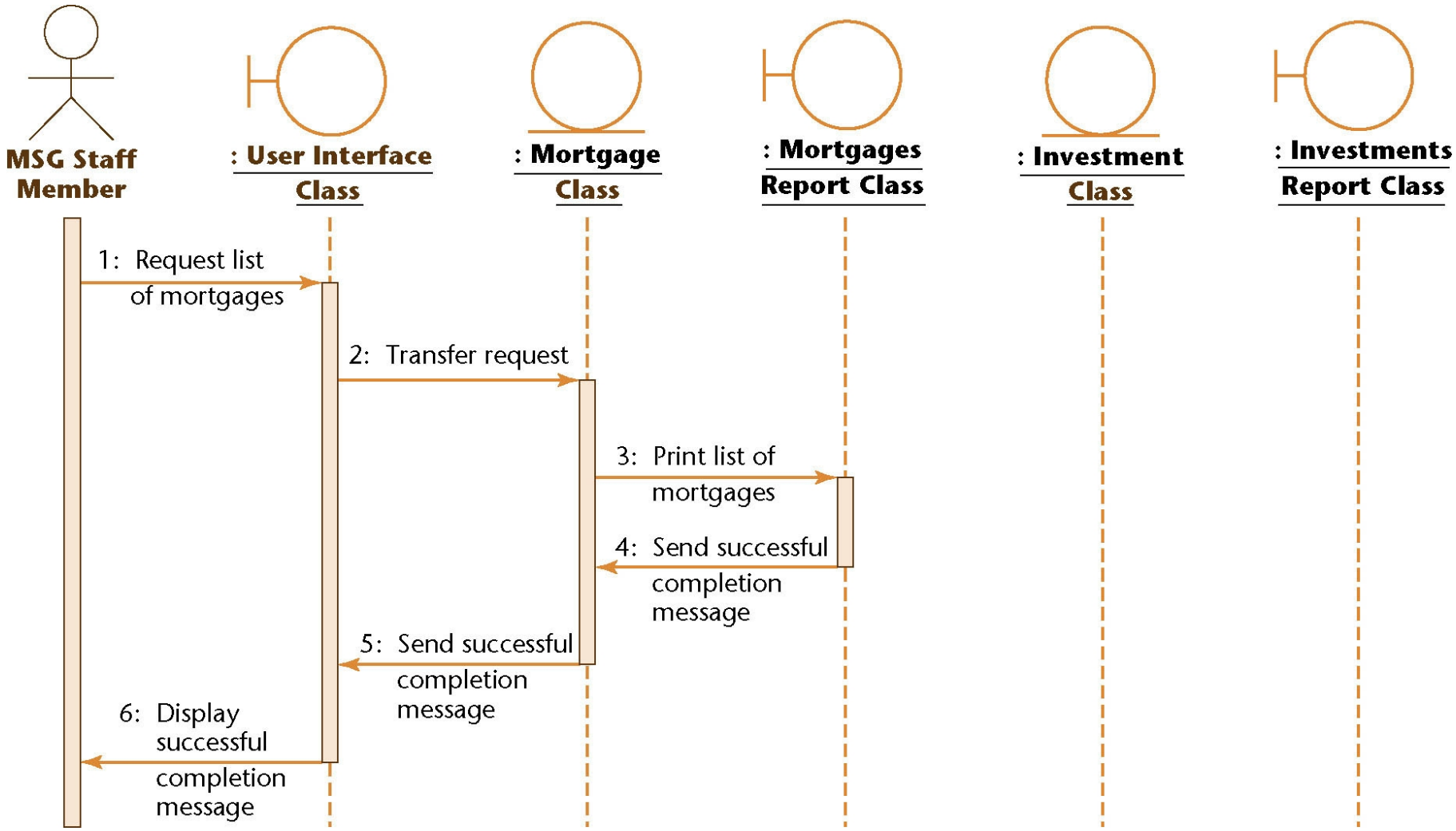
Produce a Report Use Case (contd)

- Collaboration diagram
  - Mortgages (but not investments) are involved



Produce a Report Use Case (contd)

- Sequence diagram



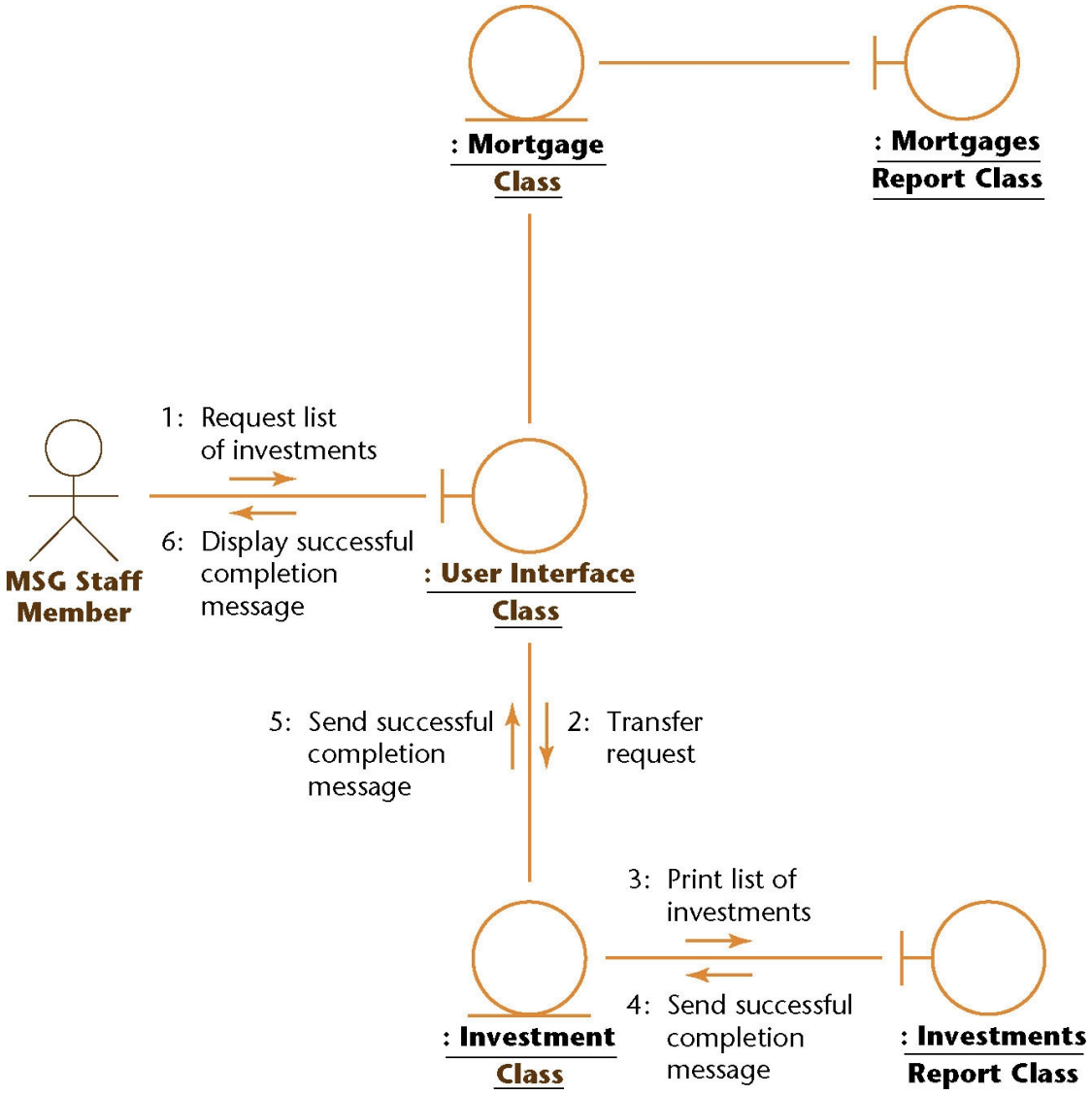
Produce a Report Use Case (contd)

- A second scenario (listing all investments) of the use case

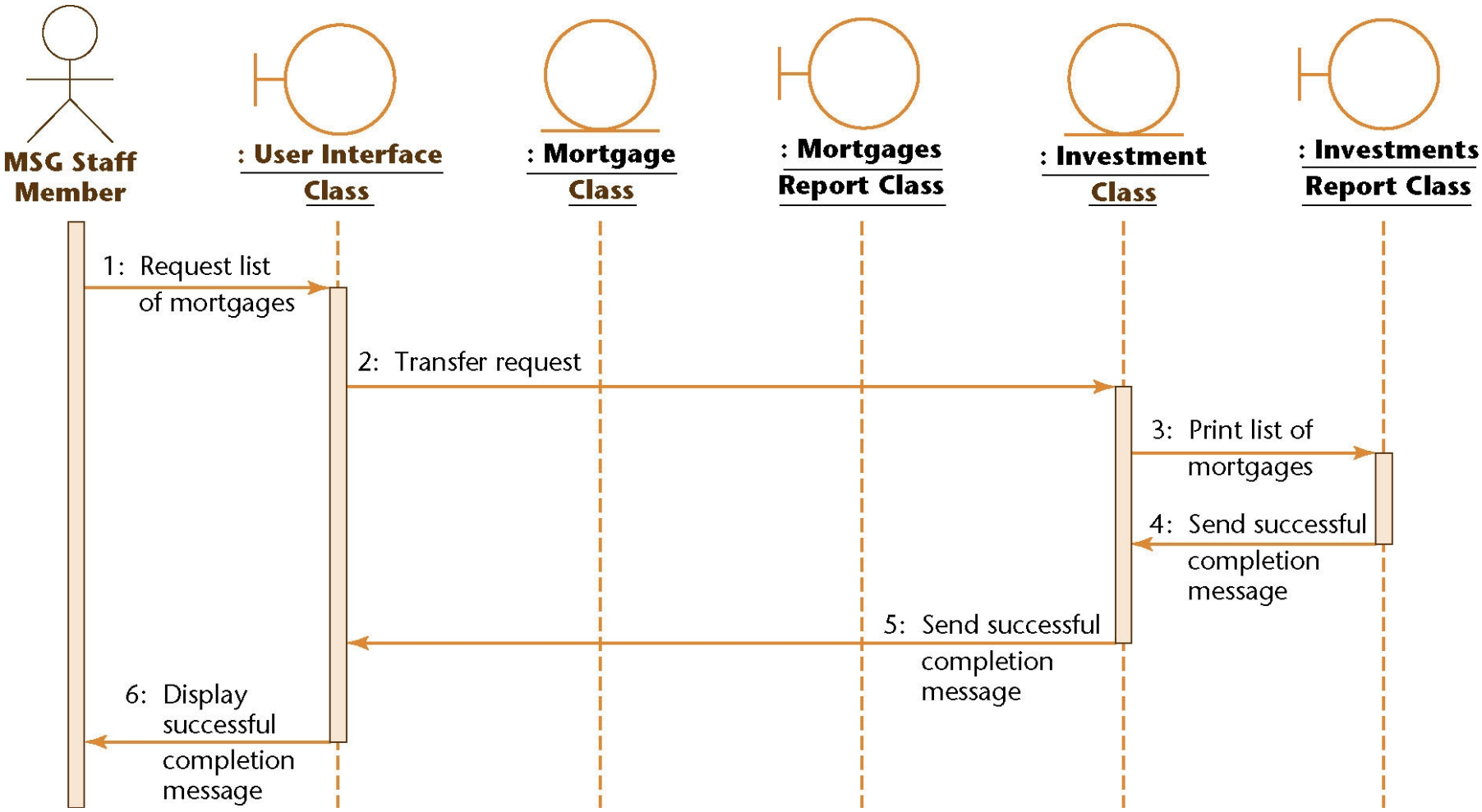
> An MSG staff member wishes to print a list of all investments.
> 1. The staff member requests a report listing all investments.

Produce a Report Use Case (contd)

- Collaboration diagram for second scenario
  - This time, investments (but not mortgages) are involved



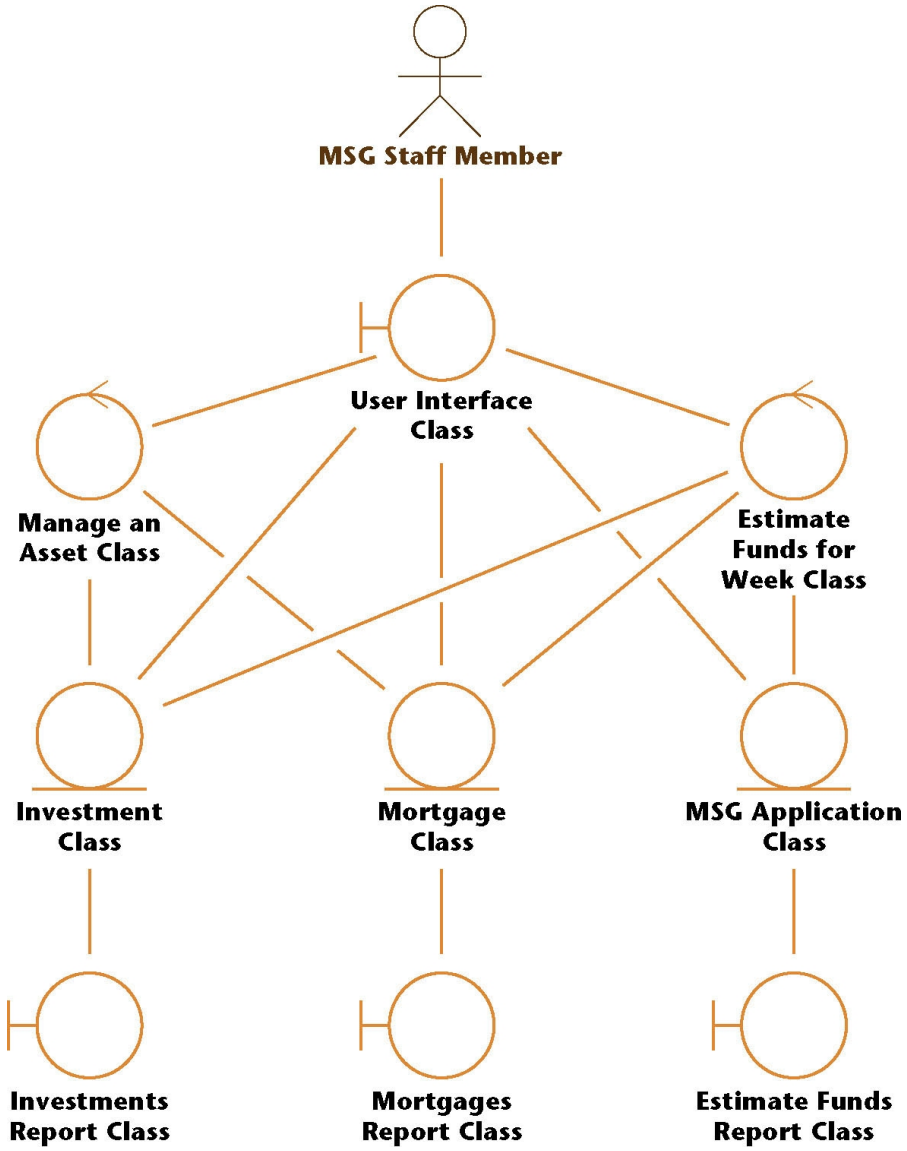
Produce a Report Use Case (contd)



12.16 Incrementing the Class Diagram: The MSG Foundation

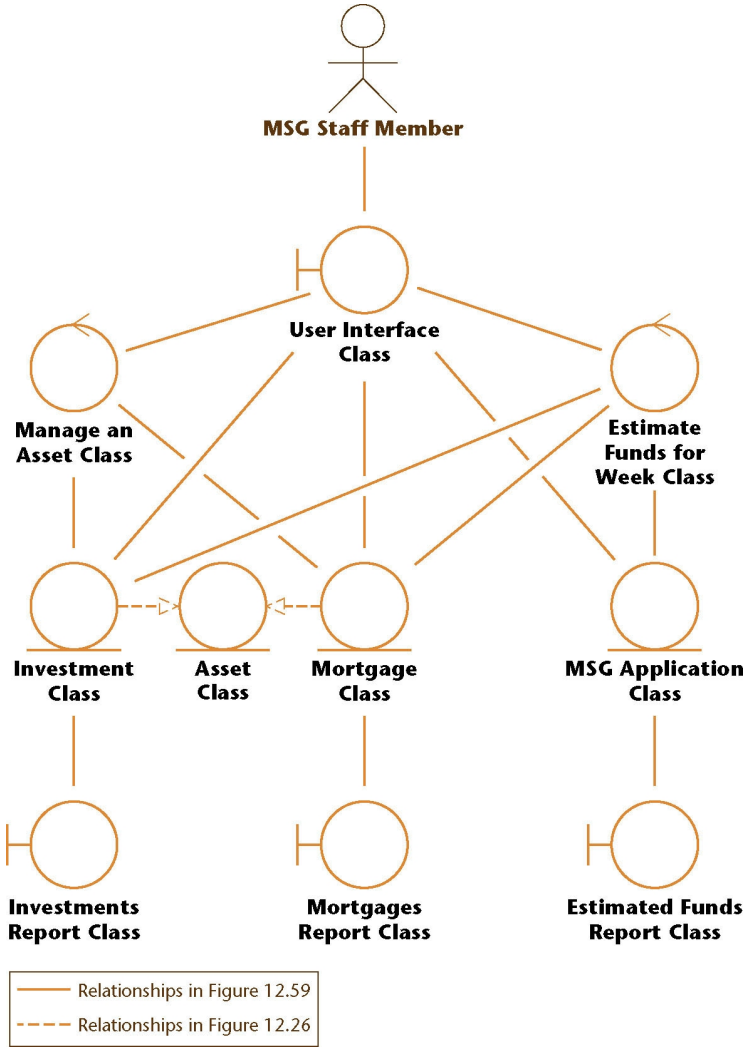
- In the course of realizing the various use cases
  - Interrelationships between classes become apparent
- Accordingly, we now combine the realization class diagrams

Combining the Realization Class Diagrams



Fourth Iteration of the Class Diagram

- Fifth iteration + realization class diagram



Software Project Management Plan

- As with the classical paradigm, the SPMP is drawn up at this point
  - It appears in Appendix F
  - The plan conforms to the IEEE SPMP format

12.17 The Test Workflow: MSG Foundation

- CRC cards are used to check the entity classes
- All the artifacts are then inspected

12.18 The Specification Document in the Unified Process

- The Unified Process is use-case driven
  - The use cases and the artifacts derived from them replace the traditional textual specification document
- The client must be shown each use case and associated artifacts, both diagrammatic and textual
  - These UML diagrams convey to the client more information more accurately than the traditional specification document
  - The set of UML diagrams can also play the same contractual role as the traditional specification document

The Specification Document (contd)

- A scenario is a specific execution sequence
- The client can therefore appreciate how the product works equally well from
  - A use case together with its scenarios, or
  - A rapid prototype
- The difference is
  - The use cases are successively refined, with more information added each time, whereas
  - The rapid prototype is discarded

The Specification Document (contd)

- However, a rapid prototype of the user interface is required
  - Specimen screens and reports are needed (not a complete rapid prototype)

12.19 More on Actors and Use Cases

- To find the actors, consider every role in which an individual can interact with the software product
  - Example: **Applicants**, **Borrowers**
- Actors are not individuals
  - They are roles played by those individuals
- Find all the different roles played by each user
  - From the list of roles, extract the actors

More on Actors and Use Cases (contd)

- In the Unified Process
  - The term *worker* is used to denote a role played by an individual
  - In the Unified Process, **Applicants** and **Borrowers** are two different workers
- In common parlance
  - The word “worker” usually refers to an employee
- In this book, the word “role” is used in place of “worker”

More on Actors and Use Cases (contd)

- Within a business context, finding the roles is easy
  - They are displayed within the use-case business model
- To find the actors
  - Find the subset of the use-case business model that corresponds to the use-case model of the requirements

More on Actors and Use Cases (contd)

- Construct the use-case business model
- Consider only those parts of the business model that correspond to the proposed software product
- The actors in this subset are the actors we seek

More on Actors and Use Cases (contd)

- Within a business context, finding use cases is easy
- For each role, there will be one or more use cases
  - Find the actors (see previous slide)
  - The use cases then follow

12.20 CASE Tools for the Object-Oriented Analysis Workflow

- Diagrams play a major role in object-oriented analysis
- Diagrams often change
  - We need a diagramming tool
  - Many tools go further
- All modern tools support UML
  - Commercial examples
    - IBM Rational Rose
    - Together
  - Open-source example
    - ArgoUML

12.21 Challenges of the Object-Oriented Analysis Workflow

- Do not cross the boundary into object-oriented design
- Do not allocate methods to classes *yet*
  - Reallocating methods to classes during stepwise refinement is wasted effort